Ok if i go only for JVM language is java a good option? Do we have good Api for msteams, slack, discord etc in Java and is it a good language for it?

Yes, **Java is an excellent choice** if you are targeting the JVM ecosystem. Writing the core library in standard Java guarantees 100% native compatibility across Kotlin, Scala, Groovy, and Clojure without imposing language-specific runtime dependencies or interop friction on your users.

## 1. Existing Java API & SDK Ecosystem

The Java ecosystem already has some of the most mature SDKs and client libraries available across major messaging platforms:

- **Slack:** Slack directly maintains the official Slack SDK for Java ( `com.slack.api:slack-api-client` ). It provides built-in support for Slack's Block Kit UI, Webhooks, REST methods, and Socket Mode.
  GitHub
- **Discord:** Java boasts one of the most active Discord developer ecosystems. JDA (Java Discord API) and **Discord4J** are enterprise-grade libraries with built-in rate-limiting queues, event systems, and REST clients.
  GitHub
- **Microsoft Teams:** Simple channel alerts in Teams rely on incoming webhooks accepting **Adaptive Cards** JSON payloads. For direct user messaging or bot-driven interactions, Microsoft provides the official **Microsoft Graph SDK for Java**.
  Microsoft
- **Telegram:** The `telegrambots` library in Java is widely used, active, and fully covers Telegram's Bot API.
- **Google Chat:** Google actively maintains the official `google-api-services-chat` Java client library.

> **Architecture Tip:** For basic channel webhooks (Slack, Discord, Teams, Mattermost), you often don't even need to import these heavy SDKs as dependencies. You can write lightweight internal JSON serializers for each platform's payload schema and handle the network calls directly.

## 2. Why Java is Uniquely Suited for This Library

Building a unified notification wrapper in modern Java ( targeting Java 17 or Java 21) offers several strategic advantages:

## Zero-Dependency Footprint

Since Java 11, the JDK includes a native `java.net.http.HttpClient` that supports HTTP/2, synchronous/asynchronous calls, and connection pooling out of the box. By relying on native JDK features, your library won't force downstream consumers to pull in third-party HTTP clients (like OkHttp or Apache HttpClient) that risk transitive dependency conflicts.

## Universal JVM Interoperability

If you write a library in Kotlin, Java users often face awkward interop syntax around coroutines, companion objects, or missing `@JvmStatic` / `@JvmOverloads` annotations. Writing in clean, standard Java ensures that:

- **Java developers** get idiomatic Builder patterns.
- **Kotlin developers** get seamless object instantiation (and you can easily ship a tiny secondary `kotlin-extensions` module with DSL builders if desired).
- **Scala developers** can invoke it without wrapper layers.

## Massive Throughput with Virtual Threads (Java 21+)

Sending outbound Webhooks to multiple chat services is an I/O-heavy operation. With Virtual Threads (Project Loom), your library can fire off hundreds of concurrent platform notifications using simple, readable synchronous code—without starving thread pools or requiring consumers to adopt reactive frameworks like RxJava or Reactor.

## Expressive Domain Modeling (Java 17 Records & Sealed Classes)

You can define your unified message domain model using Java `record` types for thread-safe, immutable data transfers, and `sealed interfaces` to strictly constrain supported platforms or card layouts.

Java

```java
// Example: Modern Java unified dispatch signature
public sealed interface ChannelDestination permits SlackChannel, TeamsChannel, Discor

public record UnifiedMessage(String title, String body, StatusColor color) {}

public record NotificationResult(boolean success, int statusCode, String platformResp
```

Give me a list with the main facing methods eg sendPrivateMessage, sendGroupMessage, create channel etc

To build an intuitive, multi-platform messaging library in Java, your main entry point (such as a `ChatClient` or `NotificationService` interface) should organize methods into logical domains.

Because messaging APIs differ across Slack, Teams, and Discord, your interface methods should accept unified request objects (or flexible IDs) and return structured responses.

## 1. Direct & Group Messaging (Core Messaging)

Methods for firing off messages to individuals, channels, or group chats.

Java

```java
public interface ChatClient {

    // Send a message to a specific user (automatically opens or reuses a DM channel)
    MessageResponse sendPrivateMessage(String userId, Message message);
    CompletableFuture<MessageResponse> sendPrivateMessageAsync(String userId, Message

    // Send a message to a public/private channel or room
    MessageResponse sendChannelMessage(String channelId, Message message);
    CompletableFuture<MessageResponse> sendChannelMessageAsync(String channelId, Mess

    // Send an ephemeral message visible ONLY to a specific user inside a shared chan
    MessageResponse sendEphemeralMessage(String channelId, String userId, Message mes

    // Update or delete previously posted messages
    MessageResponse updateMessage(String channelId, String messageId, Message newMess
    void deleteMessage(String channelId, String messageId);
}
```

## 2. Threading & Replies

Handling conversational trees and replies across platforms that support threaded conversations.

Java

```java
// Post a reply directly inside a message thread (e.g., Slack thread, Teams reply
MessageResponse sendThreadReply(String channelId, String parentMessageId, Message

// Broadcast a thread reply back into the main channel timeline (Slack "also send
MessageResponse sendThreadReplyAndBroadcast(String channelId, String parentMessag

// Fetch message history or thread replies
List<Message> getThreadReplies(String channelId, String parentMessageId);
```

## 3. Channel & Conversation Management

Methods for provisioning channels, managing members, and querying room metadata.

Java

```java
// Create a new public or private channel
ChannelInfo createChannel(CreateChannelRequest request); // Request specifies nam

// Archive or soft-delete a channel
void archiveChannel(String channelId);

// Set channel metadata
void setChannelTopic(String channelId, String topic);

// Membership management
void inviteToChannel(String channelId, List<String> userIds);
void kickFromChannel(String channelId, String userId);

// List/Lookup channels
List<ChannelInfo> listChannels(ChannelFilter filter);
ChannelInfo getChannelDetails(String channelId);
```

## 4. File & Media Attachments

Handling file uploads, embedded images, and attachments across platforms.

GitHub

Java

```
// Send a message with one or more file attachments (logs, PDFs, images)
MessageResponse sendFile(String channelId, FileUploadRequest fileRequest);

// Upload a file directly into a thread reply
MessageResponse sendThreadFile(String channelId, String parentMessageId, FileUplo
```

## 5. Reactions, Status & Ephemeral Actions

Interactions beyond text, such as emoji reactions or user indicators.

Java

```
// Add or remove emoji reactions to a message
void addReaction(String channelId, String messageId, String emojiName);
void removeReaction(String channelId, String messageId, String emojiName);

// Trigger a temporary typing indicator ("Bot is typing...")
void sendTypingIndicator(String channelId);
```

## 6. Low-Level / Platform-Specific Passthrough (Escape Hatch)

Because every platform has unique features that your abstraction won't cover 100% of the time, providing a native "escape hatch" prevents users from feeling trapped by your library.

Java

```
// Raw payload execution for platform-specific edge cases
<T> T executeRaw(String destinationPlatform, String endpoint, Object rawPayload,
```

## Example Design Pattern for `Message` Construction

To keep signatures clean, avoid taking raw `String text` everywhere. Instead, use a Fluent Builder or Java `record` for `Message`:

Java

```
Message message = Message.builder()
    .title("Deployment Alert")
    .body("Service **user-service** successfully deployed to production.")
    .color(StatusColor.GREEN)
    .addField("Cluster", "eu-west-1")
    .addField("Duration", "42s")
    .addButton("View Logs", "https://logs.internal.com")
    .build();

// Standard execution across any configured destination
chatClient.sendChannelMessage("msteams://webhook-id-123", message);
chatClient.sendChannelMessage("slack://C0123456789", message);
```

Ok i believe that all those should be based on type as well e.g. I will have the MessageClient whichbeill have MSteamsMessageClient implementation. And this will be part of MSTeamsClient object, seems good? If not tell m

Exposing platform-specific objects like `MSTeamsClient` or `SlackClient` directly to your consumers negates the main benefit of a unified library. If user code has to cast or check for `MSTeamsClient`, it creates tight coupling, and changing or adding platform destinations later requires rewriting downstream application logic.

Instead, the internal implementations should be hidden behind a unified interface (`ChatClient` or `ChatProvider`), while feature differences across platforms are handled cleanly using established API design patterns.

### Pattern 1: Interface Segregation & Optional Extensions (Type-Safe)

Split heavy features into role-based interfaces (Interface Segregation Principle). Every platform implements the base `ChatClient`, but only platforms that support specific capabilities implement extended interfaces like `ThreadSupport` or `ChannelAdmin`.

Java

```java
// Core interface supported by ALL platforms
public interface ChatClient {
    MessageResponse sendMessage(String destination, Message message);

    // Helper to query/cast optional features safely
    default <T> Optional<T> as(Class<T> featureClass) {
        return featureClass.isInstance(this) ? Optional.of(featureClass.cast(this)) :
    }
}

// Optional capabilities
public interface ThreadSupport {
    MessageResponse sendThreadReply(String channelId, String threadId, Message messag
}

public interface ChannelAdmin {
    ChannelInfo createChannel(String name, boolean isPrivate);
}
```

Consumer Usage:

Java

```java
ChatClient client = clientRegistry.get("msteams");

// Basic message always works
client.sendMessage("channel-id", message);

// Thread reply is attempted only if the platform supports it
client.as(ThreadSupport.class)
      .ifPresent(threads -> threads.sendThreadReply("channel-id", "thread-123", reply
```

## Pattern 2: Capability Query Matrix (Runtime Feature Toggling)

Provide a light capability discovery mechanism via an `Enum` set. The core interface remains unified, and clients can programmatically check what features are supported before calling

them.

Java

```java
public enum Capability {
    SEND_DIRECT_MESSAGE,
    THREAD_REPLIES,
    EPHEMERAL_MESSAGES,
    EMOJI_REACTIONS,
    CHANNEL_MANAGEMENT
}

public interface ChatClient {
    boolean supports(Capability capability);
    Set<Capability> getCapabilities();

    MessageResponse sendEphemeralMessage(String channelId, String userId, Message mes
}
```

Implementation in Provider:

Java

```java
public class SlackClientAdapter implements ChatClient {
    private static final Set<Capability> CAPABILITIES = EnumSet.of(
        Capability.SEND_DIRECT_MESSAGE,
        Capability.THREAD_REPLIES,
        Capability.EPHEMERAL_MESSAGES
    );

    @Override
    public boolean supports(Capability capability) {
        return CAPABILITIES.contains(capability);
    }

    @Override
    public MessageResponse sendEphemeralMessage(String channelId, String userId, Mess
        if (!supports(Capability.EPHEMERAL_MESSAGES)) {
            throw new UnsupportedOperationException("Ephemeral messages are not suppo
        }
        // Slack implementation...
```

```
    }
}
```

### Pattern 3: Graceful Degradation (Polyfill Strategy)

For visual features (such as buttons, cards, status colors, or embeds), avoid throwing exceptions when a feature isn't supported. Instead, make the platform adapter degrade gracefully.

| Feature Requested | Supported Platform (e.g. Slack/Teams) | Non-Supported Platform (e.g. Simple Webhook) |
|---|---|---|
| **Status Color: RED** | Formats block border/card as red | Prepends 🚨 `[ALERT]` to text body |
| **Action Button** | Renders native interactive button | Appends markdown link `[View Logs](url)` |
| **Ephemeral Message** | Native hidden message | Sends standard channel message or private DM |

...

### Recommended Architecture Summary

1. **Keep 1 Primary Client:** Consumers should interact exclusively with `ChatClient`.
2. **Hide Providers:** Implementations (`SlackClientAdapter`, `TeamsClientAdapter`) remain package-private inside your library.
3. **Use Interface Segregation (`.as(Capability.class)`)** for heavy operations like channel administration or thread creation.
4. **Use Graceful Degradation** inside payload builders for formatting differences so messages always render cleanly regardless of destination.

Can you create an md file of how i can test each one? Note that i dont have slack, teams etc so i can create whatever needed. I want all the steps from administration to the configuration to the java test, what is expected what err

# Integration Testing Guide: Unified Messaging Library

This guide provides step-by-step instructions to set up, configure, and test integrations for **Discord**, **Telegram**, **Slack**, **Google Chat**, and **Microsoft Teams** from scratch.

The platforms are ordered from **easiest setup to most complex**.

## 1. Discord

### Administration & Webhook Setup (~1 minute)

1. Go to discord.com and log in or register a free account.
2. On the left sidebar, click **+** (**Add a Server**) → **Create My Own** → **For me and my friends**. Name it `Test Server`.
3. In your new server, hover over the `#general` text channel and click the **Gear Icon** (**Edit Channel**).
4. Select **Integrations** → **Webhooks** → **New Webhook**.
5. Set a name (e.g., `Integration Tester`) and click **Copy Webhook URL**.

### Configuration Parameters

Properties

```
DISCORD_WEBHOOK_URL=https://discord.com/api/webhooks/{webhook_id}/{webhook_token}
```

### Fast Validation (cURL)

Bash

```
curl -H "Content-Type: application/json" \
     -X POST \
     -d '{"content": "Hello from Discord Test!"}' \
     "YOUR_DISCORD_WEBHOOK_URL"
```

## Java Integration Test

Java

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class DiscordTest {
    private static final String WEBHOOK_URL = "YOUR_DISCORD_WEBHOOK_URL";

    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newHttpClient();
        String jsonPayload = """
            {
                "content": "Hello from Java Unified Messaging Library!"
            }
            """;

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(WEBHOOK_URL))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(jsonPayload))
                .build();

        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandler
        System.out.println("HTTP Status Code: " + response.statusCode());
    }
}
```

## Expected Output

- **HTTP Status:** `204 No Content`
- Message appears instantly in the `#general` Discord channel.

## Common Errors & Troubleshooting

| Error Code | Cause | Resolution |
| --- | --- | --- |
| `400 Bad Request` | Missing required `content` field or malformed JSON. | Ensure body has `"content": "text"`. |

| Error Code | Cause | Resolution |
| --- | --- | --- |
| `404 Not Found` | Invalid Webhook ID or Token in the URL. | Re-copy URL from Channel Settings → Integrations. |
| `429 Too Many Requests` | Rate limit exceeded (Discord limits to 30 requests/30s). | Parse `Retry-After` header and wait before retrying. |

...

## 2. Telegram

### Administration & Bot Setup (~2 minutes)

1. Open Telegram (Desktop/Mobile) and search for `@BotFather` (verified blue checkmark).
2. Send the command `/newbot` to `@BotFather`.
3. Enter a display name (e.g., `Test Notification Bot`) and a unique username ending in `bot` (e.g., `my_test_java_bot`).
4. `@BotFather` will generate an **HTTP API Token** (e.g., `123456789:ABCdefGHIjklmNOPqrstUVwxyz`). Save this token.
5. Create a new Telegram Group or Channel, and add your bot as a member.
6. Send a dummy message (e.g., `test`) into the group.
7. Find your `CHAT_ID` by opening this URL in your browser:

   `[https://api.telegram.org/bot](https://api.telegram.org/bot)<YOUR_BOT_TOKEN>/getUpdates`

   Look for `"chat":{"id": -100xxxxxxxxx}` in the JSON response.

### Configuration Parameters

**Properties**

```
TELEGRAM_BOT_TOKEN=123456789:ABCdefGHIjklmNOPqrstUVwxyz
TELEGRAM_CHAT_ID=-1001234567890
```

### Fast Validation (cURL)

Bash

```bash
curl -X POST "https://api.telegram.org/bot<YOUR_BOT_TOKEN>/sendMessage" \
     -H "Content-Type: application/json" \
     -d '{"chat_id": "<YOUR_CHAT_ID>", "text": "Hello from Telegram Test!"}'
```

## Java Integration Test

Java

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class TelegramTest {
    private static final String BOT_TOKEN = "YOUR_BOT_TOKEN";
    private static final String CHAT_ID = "YOUR_CHAT_ID";

    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newHttpClient();
        String url = "https://api.telegram.org/bot" + BOT_TOKEN + "/sendMessage";

        String jsonPayload = String.format("""
            {
                "chat_id": "%s",
                "text": "Hello from Java Unified Messaging Library!"
            }
            """, CHAT_ID);

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(jsonPayload))
                .build();

        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandler
        System.out.println("HTTP Status Code: " + response.statusCode());
        System.out.println("Body: " + response.body());
    }
}
```

### Expected Output

- **HTTP Status:** `200 OK`
- **Body:** `{"ok":true,"result":{...}}`
- Message appears in the Telegram group/channel.

### Common Errors & Troubleshooting

| Error Code | Cause | Resolution |
|---|---|---|
| `401 Unauthorized` | Invalid Bot Token. | Double-check token string provided by `@BotFather`. |
| `400 Bad Request` | `chat_id` not found or invalid format. | Ensure group ID includes negative prefix (e.g. `-100...`). |
| `403 Forbidden` | Bot was blocked or removed from group. | Re-add bot to group/channel and assign admin permissions. |

`...`

## 3. Slack

### Administration & Webhook Setup (~3 minutes)

1. Go to slack.com and create a free test workspace.
2. Navigate to api.slack.com/apps and click **Create New App** → **From scratch**.
3. Name your app `Test Bot` and pick your workspace.
4. Under **Features**, click **Incoming Webhooks**.
5. Toggle **Activate Incoming Webhooks** to **On**.
6. Click **Add New Webhook to Workspace** at the bottom. Select `#general` and click **Allow**.
7. Copy the generated **Webhook URL** (starts with `[https://hooks.slack.com/services/](https://hooks.slack.com/services/)...`).

### Configuration Parameters

Properties

```
SLACK_WEBHOOK_URL=https://hooks.slack.com/services/T00000000/B00000000/XXXXXXXXXXXXXX
```

## Fast Validation (cURL)

Bash

```
curl -X POST -H 'Content-type: application/json' \
     --data '{"text":"Hello from Slack Test!"}' \
     YOUR_SLACK_WEBHOOK_URL
```

## Java Integration Test

Java

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class SlackTest {
    private static final String WEBHOOK_URL = "YOUR_SLACK_WEBHOOK_URL";

    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newHttpClient();
        String jsonPayload = """
            {
                "text": "Hello from Java Unified Messaging Library!"
            }
            """;

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(WEBHOOK_URL))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(jsonPayload))
                .build();

        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandler
```

```
        System.out.println("HTTP Status Code: " + response.statusCode());
        System.out.println("Body: " + response.body());
    }
}
```

### Expected Output

- **HTTP Status:** `200 OK`
- **Body:** `ok`
- Message is posted to the selected Slack channel.

### Common Errors & Troubleshooting

| Error Code | Cause | Resolution |
|---|---|---|
| `404 Not Found` | Invalid Webhook URL. | Regenerate Webhook in app settings. |
| `400 Bad Request` | Missing `text` field in root JSON. | Ensure body contains `"text": "..."` field. |
| `403 Forbidden` | Channel was deleted or archived. | Re-link webhook to an active channel. |

`...`

## 4. Google Chat

### Administration & Space Setup (~3 minutes)

1. Open Google Chat (chat.google.com).
2. Click **+** next to Spaces → **Create a space**. Name it `Test Space`.
3. Click the Space name at the top → **Apps & integrations** (or **Manage webhooks**).
4. Click **Add webhooks**.
5. Set Name to `Test Webhook` and click **Save**.
6. Copy the generated **Webhook URL** (contains `key` and `token` query parameters).

### Configuration Parameters

Properties

```
GOOGLE_CHAT_WEBHOOK_URL=https://chat.googleapis.com/v1/spaces/SPACE_ID/messages?key=K
```

## Fast Validation (cURL)

Bash

```
curl -X POST -H "Content-Type: application/json; charset=UTF-8" \
     -d '{"text": "Hello from Google Chat Test!"}' \
     "YOUR_GOOGLE_CHAT_WEBHOOK_URL"
```

## Java Integration Test

Java

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class GoogleChatTest {
    private static final String WEBHOOK_URL = "YOUR_GOOGLE_CHAT_WEBHOOK_URL";

    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newHttpClient();
        String jsonPayload = """
            {
                "text": "Hello from Java Unified Messaging Library!"
            }
            """;

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(WEBHOOK_URL))
                .header("Content-Type", "application/json; charset=UTF-8")
                .POST(HttpRequest.BodyPublishers.ofString(jsonPayload))
                .build();

        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandler
```

```
            System.out.println("HTTP Status Code: " + response.statusCode());
            System.out.println("Body: " + response.body());
        }
    }
```

### Expected Output

- **HTTP Status:** `200 OK`
- **Body:** Full message object metadata in JSON format.
- Message displays in Google Chat Space.

### Common Errors & Troubleshooting

| Error Code | Cause | Resolution |
| --- | --- | --- |
| `400 Bad Request` | Malformed JSON schema. | Validate JSON format and ensure content header includes `charset=UTF-8`. |
| `404 Not Found` | Invalid Space ID or expired URL parameters. | Re-copy webhook link from Google Chat Space settings. |
| `403 Permission Denied` | External webhooks disabled by Admin. | Enable incoming webhooks in Google Workspace Admin console. |

`...`

## 5. Microsoft Teams

> **Note on Modern Teams Webhooks:** Microsoft retired legacy Office 365 Connectors in favor of **Workflows (Power Automate)** webhooks.

### Administration & Workflow Setup (~5 minutes)

1. Open Microsoft Teams and create a team or open an existing channel (e.g., `General`).
2. Click `...` next to the channel name → **Workflows**.
3. Search for template: **"Post to a channel when a webhook request is received"**.

4. Select your **Team** and **Channel**, then click **Add Workflow / Save.**
5. Copy the generated Webhook URL (starts with `[https://...logic.azure.com/](https://...logic.azure.com/)...` or similar Azure endpoint).

## Configuration Parameters

Properties

```
MSTEAMS_WORKFLOW_WEBHOOK_URL=https://prod-xx.location.logic.azure.com:443/workflows/.
```

## Fast Validation (cURL)

Teams Workflows require an **Adaptive Card** payload structure:

Bash

```
curl -X POST -H 'Content-Type: application/json' \
     -d '{
       "type": "message",
       "attachments": [
         {
           "contentType": "application/vnd.microsoft.card.adaptive",
           "content": {
             "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
             "type": "AdaptiveCard",
             "version": "1.4",
             "body": [
               {
                 "type": "TextBlock",
                 "text": "Hello from MS Teams Workflows Test!"
               }
             ]
           }
         }
       ]
     }' \
     "YOUR_TEAMS_WORKFLOW_WEBHOOK_URL"
```

## Java Integration Test

Java

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class MSTeamsTest {
    private static final String WEBHOOK_URL = "YOUR_TEAMS_WORKFLOW_WEBHOOK_URL";

    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newHttpClient();
        String jsonPayload = """
            {
               "type": "message",
               "attachments": [
                 {
                   "contentType": "application/vnd.microsoft.card.adaptive",
                   "content": {
                     "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
                     "type": "AdaptiveCard",
                     "version": "1.4",
                     "body": [
                       {
                         "type": "TextBlock",
                         "text": "Hello from Java Unified Messaging Library!"
                       }
                     ]
                   }
                 }
               ]
             }
            """;

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(WEBHOOK_URL))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(jsonPayload))
                .build();

        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandler
        System.out.println("HTTP Status Code: " + response.statusCode());
    }
}
```

## Expected Output

- **HTTP Status:** `202 Accepted` or `200 OK`
- Formatted Adaptive Card message appears inside Microsoft Teams channel.

## Common Errors & Troubleshooting

| Error Code | Cause | Resolution |
| --- | --- | --- |
| `400 Bad Request` | Invalid Adaptive Card schema. | Ensure `$schema`, `version`, `type`, and `attachments` structure match Adaptive Card v1.4. |
| `403 Forbidden` | Workflow trigger turned off or IP policy blocking call. | Check Power Automate status for the workflow under "My Flows". |
| `404 Not Found` | Workflow deleted or URL path truncated. | Re-copy webhook endpoint from Teams Workflows tab. |

...

Like i said, i want all those infos to be in an md file. Also, we should have e2e test which means create channel, send message there etc. The administration steps should cover all the possibilities for CRUD actions plus media and replies to threads

Markdown

```
# Full End-to-End (E2E) Integration Testing Guide

This document provides complete administration setup steps, permission requirements,

The guide covers full **CRUD operations** (Channel management, Sending/Updating/Delet
```

````
---

## 1. Discord

### Administration & Permissions Setup

1. **Create Application & Bot:**
   * Go to the [Discord Developer Portal](https://discord.com/developers/applications
   * Click **New Application**, name it `E2E Test Bot`, and click **Create**.
   * Go to the **Bot** tab on the left menu and click **Reset Token** to generate a n

2. **Configure Bot Permissions:**
   * In the **Bot** tab, enable the following **Privileged Gateway Intents**:
     * **Message Content Intent** (required to read message payloads during tests).
   * Go to **OAuth2** $\rightarrow$ **URL Generator**.
   * Under **Scopes**, select `bot`.
   * Under **Bot Permissions**, select:
     * **Manage Channels** (Create/Edit/Delete channels)
     * **Send Messages**
     * **Send Messages in Threads**
     * **Create Public Threads**
     * **Read Message History**
     * **Attach Files**
     * **Manage Messages** (Update/Delete messages)

3. **Invite Bot to Server:**
   * Copy the generated OAuth2 URL at the bottom of the page.
   * Open the URL in a browser, select your Discord Test Server, and click **Authoriz

---

### End-to-End Java Integration Test

```java
import java.io.IOException;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class DiscordE2ETest {
    private static final String BOT_TOKEN = System.getenv("DISCORD_BOT_TOKEN");
    private static final String GUILD_ID = System.getenv("DISCORD_GUILD_ID"); // Serv
    private static final HttpClient client = HttpClient.newHttpClient();

    public static void main(String[] args) throws Exception {
        // 1. CREATE CHANNEL
        String channelId = createChannel("e2e-test-channel");
        System.out.println("Created Channel ID: " + channelId);
```
````

```
        // 2. SEND CHANNEL MESSAGE
        String messageId = sendMessage(channelId, "Hello from E2E Test!");
        System.out.println("Sent Message ID: " + messageId);

        // 3. THREAD REPLY
        String threadId = createThreadAndReply(channelId, messageId, "This is a threa
        System.out.println("Created Thread ID: " + threadId);

        // 4. UPDATE MESSAGE
        updateMessage(channelId, messageId, "Updated E2E Message Content");
        System.out.println("Updated Message successfully.");

        // 5. UPLOAD MEDIA
        uploadFile(channelId, "test.txt", "Sample file content for Discord upload.");
        System.out.println("Uploaded Media successfully.");

        // 6. DELETE MESSAGE & CLEANUP CHANNEL
        deleteMessage(channelId, messageId);
        deleteChannel(channelId);
        System.out.println("E2E Cleanup Complete.");
    }

    private static String createChannel(String name) throws Exception {
        String json = String.format("{\"name\": \"%s\", \"type\": 0}", name);
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("[https://discord.com/api/v10/guilds/](https://discor
                .header("Authorization", "Bot " + BOT_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json))
                .build();

        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandler
        return extractJsonValue(response.body(), "id");
    }

    private static String sendMessage(String channelId, String text) throws Exception
        String json = String.format("{\"content\": \"%s\"}", text);
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("[https://discord.com/api/v10/channels/](https://disc
                .header("Authorization", "Bot " + BOT_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json))
                .build();

        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandler
        return extractJsonValue(response.body(), "id");
    }
```

```
    private static String createThreadAndReply(String channelId, String messageId, St
        String json = String.format("{\"name\": \"E2E Thread\", \"auto_archive_durati
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("[https://discord.com/api/v10/channels/](https://disc
                .header("Authorization", "Bot " + BOT_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json))
                .build();

        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandler
        String threadId = extractJsonValue(response.body(), "id");

        // Reply inside the newly created thread
        sendMessage(threadId, text);
        return threadId;
    }

    private static void updateMessage(String channelId, String messageId, String newT
        String json = String.format("{\"content\": \"%s\"}", newText);
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("[https://discord.com/api/v10/channels/](https://disc
                .header("Authorization", "Bot " + BOT_TOKEN)
                .header("Content-Type", "application/json")
                .method("PATCH", HttpRequest.BodyPublishers.ofString(json))
                .build();

        client.send(request, HttpResponse.BodyHandlers.ofString());
    }

    private static void uploadFile(String channelId, String filename, String content)
        String boundary = "----DiscordBoundary" + System.currentTimeMillis();
        String body = "--" + boundary + "\r\n" +
                "Content-Disposition: form-data; name=\"files[0]\"; filename=\"" + fi
                "Content-Type: text/plain\r\n\r\n" +
                content + "\r\n" +
                "--" + boundary + "--\r\n";

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("[https://discord.com/api/v10/channels/](https://disc
                .header("Authorization", "Bot " + BOT_TOKEN)
                .header("Content-Type", "multipart/form-data; boundary=" + boundary)
                .POST(HttpRequest.BodyPublishers.ofString(body))
                .build();

        client.send(request, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteMessage(String channelId, String messageId) throws Exce
        HttpRequest request = HttpRequest.newBuilder()
```

```
                .uri(URI.create("[https://discord.com/api/v10/channels/](https://disc
                .header("Authorization", "Bot " + BOT_TOKEN)
                .DELETE()
                .build();

        client.send(request, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteChannel(String channelId) throws Exception {
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("[https://discord.com/api/v10/channels/](https://disc
                .header("Authorization", "Bot " + BOT_TOKEN)
                .DELETE()
                .build();

        client.send(request, HttpResponse.BodyHandlers.ofString());
    }

    private static String extractJsonValue(String json, String key) {
        String pattern = "\"" + key + "\":\"";
        int start = json.indexOf(pattern) + pattern.length();
        if (start < pattern.length()) {
            pattern = "\"" + key + "\":";
            start = json.indexOf(pattern) + pattern.length();
            int end = json.indexOf(",", start);
            if (end == -1) end = json.indexOf("}", start);
            return json.substring(start, end).trim();
        }
        int end = json.indexOf("\"", start);
        return json.substring(start, end);
    }
}
```

## 2. Telegram

### Administration & Permissions Setup

1. **Create Telegram Bot:**
    1. Open Telegram and search for `@BotFather`.
    2. Send `/newbot`, choose a name and username for your bot.
    3. Save the generated **HTTP API Token** (`TELEGRAM_BOT_TOKEN`).
2. **Configure Group Permissions for CRUD & Topics:**
    1. Create a Telegram **Supergroup** (Convert a standard group to a Supergroup by enabling **Topics** in Group Settings).

2. Add your bot to the Supergroup.
3. Grant the Bot **Admin Permissions**:
    1. *Manage Topics* (required for Thread support)
    2. *Post Messages*
    3. *Edit Messages of Others*
    4. *Delete Messages of Others*
    5. *Pin Messages*
4. Disable Privacy Mode via `@BotFather`: Send `/setprivacy` → Select your Bot → Choose **Disable**.

## End-to-End Java Integration Test

Java

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class TelegramE2ETest {
    private static final String BOT_TOKEN = System.getenv("TELEGRAM_BOT_TOKEN");
    private static final String CHAT_ID = System.getenv("TELEGRAM_CHAT_ID"); // Super
    private static final HttpClient client = HttpClient.newHttpClient();

    public static void main(String[] args) throws Exception {
        // 1. CREATE TOPIC (Equivalent to Channel/Thread in Telegram Supergroups)
        String threadId = createForumTopic("E2E Test Topic");
        System.out.println("Created Topic/Thread ID: " + threadId);

        // 2. SEND MESSAGE IN TOPIC
        String messageId = sendMessage(CHAT_ID, threadId, "Hello in Telegram Topic!")
        System.out.println("Sent Message ID: " + messageId);

        // 3. EDIT MESSAGE
        editMessage(CHAT_ID, messageId, "Updated Telegram Message");
        System.out.println("Edited Message successfully.");

        // 4. SEND DOCUMENT/MEDIA
        sendDocument(CHAT_ID, threadId, "test.txt", "Sample document content");
        System.out.println("Sent Document successfully.");

        // 5. DELETE MESSAGE & TOPIC
        deleteMessage(CHAT_ID, messageId);
```

```
        closeForumTopic(CHAT_ID, threadId);
        System.out.println("E2E Cleanup Complete.");
    }

    private static String createForumTopic(String name) throws Exception {
        String url = "[https://api.telegram.org/bot](https://api.telegram.org/bot)" +
        String json = String.format("{\"chat_id\": \"%s\", \"name\": \"%s\"}", CHAT_I

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json))
                .build();

        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandler
        return extractNestedJsonValue(response.body(), "message_thread_id");
    }

    private static String sendMessage(String chatId, String threadId, String text) th
        String url = "[https://api.telegram.org/bot](https://api.telegram.org/bot)" +
        String json = String.format("{\"chat_id\": \"%s\", \"message_thread_id\": %s,

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json))
                .build();

        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandler
        return extractNestedJsonValue(response.body(), "message_id");
    }

    private static void editMessage(String chatId, String messageId, String newText)
        String url = "[https://api.telegram.org/bot](https://api.telegram.org/bot)" +
        String json = String.format("{\"chat_id\": \"%s\", \"message_id\": %s, \"text

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json))
                .build();

        client.send(request, HttpResponse.BodyHandlers.ofString());
    }

    private static void sendDocument(String chatId, String threadId, String filename,
        String url = "[https://api.telegram.org/bot](https://api.telegram.org/bot)" +
        String boundary = "----TelegramBoundary" + System.currentTimeMillis();
```

```
        String body = "--" + boundary + "\r\n" +
                "Content-Disposition: form-data; name=\"chat_id\"\r\n\r\n" + chatId +
                "--" + boundary + "\r\n" +
                "Content-Disposition: form-data; name=\"message_thread_id\"\r\n\r\n"
                "--" + boundary + "\r\n" +
                "Content-Disposition: form-data; name=\"document\"; filename=\"" + fi
                "Content-Type: text/plain\r\n\r\n" + content + "\r\n" +
                "--" + boundary + "--\r\n";

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .header("Content-Type", "multipart/form-data; boundary=" + boundary)
                .POST(HttpRequest.BodyPublishers.ofString(body))
                .build();

        client.send(request, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteMessage(String chatId, String messageId) throws Excepti
        String url = "[https://api.telegram.org/bot](https://api.telegram.org/bot)" +
        String json = String.format("{\"chat_id\": \"%s\", \"message_id\": %s}", chat

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json))
                .build();

        client.send(request, HttpResponse.BodyHandlers.ofString());
    }

    private static void closeForumTopic(String chatId, String threadId) throws Except
        String url = "[https://api.telegram.org/bot](https://api.telegram.org/bot)" +
        String json = String.format("{\"chat_id\": \"%s\", \"message_thread_id\": %s}

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json))
                .build();

        client.send(request, HttpResponse.BodyHandlers.ofString());
    }

    private static String extractNestedJsonValue(String json, String key) {
        String pattern = "\"" + key + "\":";
        int start = json.indexOf(pattern) + pattern.length();
        int end = json.indexOf(",", start);
        if (end == -1) end = json.indexOf("}", start);
```

```
        return json.substring(start, end).trim();
    }
}
```

## 3. Slack

### Administration & Permissions Setup

1. **Create Slack App:**
    1. Go to [api.slack.com/apps](api.slack.com/apps) → **Create New App** → **From scratch**.
    2. Name your app `E2E Test Integration` and attach it to your workspace.
2. **Configure Bot Token Scopes:**
    1. Go to **OAuth & Permissions** in the sidebar.
    2. Scroll down to **Bot Token Scopes** and add the following permissions:
        1. `channels:manage` (Create, edit, archive public channels)
        2. `channels:read` (View public channels)
        3. `channels:history` (Read message history for threads)
        4. `chat:write` (Send, update, and delete messages)
        5. `files:write` (Upload files and media)
3. **Install App & Obtain Token:**
    1. Scroll to top of **OAuth & Permissions** and click **Install to Workspace**.
    2. Copy the **Bot User OAuth Token** (starts with `xoxb-...`).

### End-to-End Java Integration Test

Java

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class SlackE2ETest {
    private static final String BOT_TOKEN = System.getenv("SLACK_BOT_TOKEN");
    private static final HttpClient client = HttpClient.newHttpClient();

    public static void main(String[] args) throws Exception {
        // 1. CREATE PUBLIC CHANNEL
```

```
        String channelId = createChannel("e2e-slack-test");
        System.out.println("Created Channel ID: " + channelId);

        // 2. SEND MESSAGE
        String messageTs = sendMessage(channelId, "Hello Slack E2E!", null);
        System.out.println("Message Timestamp (ID): " + messageTs);

        // 3. THREAD REPLY
        String replyTs = sendMessage(channelId, "Thread reply to parent message", mes
        System.out.println("Reply Timestamp: " + replyTs);

        // 4. UPDATE MESSAGE
        updateMessage(channelId, messageTs, "Updated Slack Message Content");
        System.out.println("Updated Message successfully.");

        // 5. UPLOAD FILE
        uploadFile(channelId, "test.txt", "Sample upload content for Slack.");
        System.out.println("Uploaded File successfully.");

        // 6. DELETE MESSAGE & ARCHIVE CHANNEL
        deleteMessage(channelId, messageTs);
        archiveChannel(channelId);
        System.out.println("E2E Cleanup Complete.");
    }

    private static String createChannel(String name) throws Exception {
        String json = String.format("{\"name\": \"%s\"}", name);
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("[https://slack.com/api/conversations.create](https:/
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "application/json; charset=utf-8")
                .POST(HttpRequest.BodyPublishers.ofString(json))
                .build();

        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandler
        return extractNestedJsonValue(response.body(), "id");
    }

    private static String sendMessage(String channelId, String text, String threadTs)
        String json = threadTs == null ?
                String.format("{\"channel\": \"%s\", \"text\": \"%s\"}", channelId, t
                String.format("{\"channel\": \"%s\", \"text\": \"%s\", \"thread_ts\":

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("[https://slack.com/api/chat.postMessage](https://sla
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "application/json; charset=utf-8")
                .POST(HttpRequest.BodyPublishers.ofString(json))
                .build();
```

```
        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandler
        return extractNestedJsonValue(response.body(), "ts");
    }

    private static void updateMessage(String channelId, String ts, String newText) th
        String json = String.format("{\"channel\": \"%s\", \"ts\": \"%s\", \"text\":
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("[https://slack.com/api/chat.update](https://slack.co
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "application/json; charset=utf-8")
                .POST(HttpRequest.BodyPublishers.ofString(json))
                .build();

        client.send(request, HttpResponse.BodyHandlers.ofString());
    }

    private static void uploadFile(String channelId, String filename, String content)
        String boundary = "----SlackBoundary" + System.currentTimeMillis();
        String body = "--" + boundary + "\r\n" +
                "Content-Disposition: form-data; name=\"channels\"\r\n\r\n" + channel
                "--" + boundary + "\r\n" +
                "Content-Disposition: form-data; name=\"content\"\r\n\r\n" + content
                "--" + boundary + "\r\n" +
                "Content-Disposition: form-data; name=\"filename\"\r\n\r\n" + filenam
                "--" + boundary + "--\r\n";

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("[https://slack.com/api/files.upload](https://slack.c
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "multipart/form-data; boundary=" + boundary)
                .POST(HttpRequest.BodyPublishers.ofString(body))
                .build();

        client.send(request, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteMessage(String channelId, String ts) throws Exception {
        String json = String.format("{\"channel\": \"%s\", \"ts\": \"%s\"}", channelI
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("[https://slack.com/api/chat.delete](https://slack.co
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "application/json; charset=utf-8")
                .POST(HttpRequest.BodyPublishers.ofString(json))
                .build();

        client.send(request, HttpResponse.BodyHandlers.ofString());
    }
```

```
    private static void archiveChannel(String channelId) throws Exception {
        String json = String.format("{\"channel\": \"%s\"}", channelId);
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("[https://slack.com/api/conversations.archive](https:
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "application/json; charset=utf-8")
                .POST(HttpRequest.BodyPublishers.ofString(json))
                .build();

        client.send(request, HttpResponse.BodyHandlers.ofString());
    }

    private static String extractNestedJsonValue(String json, String key) {
        String pattern = "\"" + key + "\":\"";
        int start = json.indexOf(pattern) + pattern.length();
        int end = json.indexOf("\"", start);
        return json.substring(start, end);
    }
}
```

# 4. Google Chat

## Administration & Permissions Setup

1. **Configure Google Cloud Platform (GCP) Project:**
    1. Open Google Cloud Console and create a new project.
    2. Go to **APIs & Services** → **Library**, search for **Google Chat API**, and click **Enable**.
2. **Create Service Account & Credentials:**
    1. Go to **IAM & Admin** → **Service Accounts** → **Create Service Account**.
    2. Assign the role **Project Owner** or **Chat App Integration User**.
    3. Under **Keys**, click **Add Key** → **Create new key** → Choose **JSON**. Download the service account JSON key file ( `service-account.json` ).
3. **Enable OAuth2 Scopes:**
   Ensure your Service Account authenticates with the required Google Chat REST scopes:
    1. `https://www.googleapis.com/auth/chat.spaces` (Space creation and administration)
    2. `https://www.googleapis.com/auth/chat.messages` (Post, update, delete messages, and handle threads/attachments)

## End-to-End Java Integration Test

Java

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class GoogleChatE2ETest {
    private static final String ACCESS_TOKEN = System.getenv("GOOGLE_CHAT_OAUTH_TOKEN
    private static final HttpClient client = HttpClient.newHttpClient();

    public static void main(String[] args) throws Exception {
        // 1. CREATE SPACE (CHANNEL)
        String spaceId = createSpace("E2E Test Space");
        System.out.println("Created Space Name/ID: " + spaceId);

        // 2. SEND MESSAGE
        String messageName = sendMessage(spaceId, "Hello Google Chat E2E!", null);
        System.out.println("Created Message Name: " + messageName);

        // 3. THREAD REPLY
        String threadReplyName = sendMessage(spaceId, "Thread reply in Google Chat",
        System.out.println("Thread Reply Posted: " + threadReplyName);

        // 4. UPDATE MESSAGE
        updateMessage(messageName, "Updated Google Chat Message");
        System.out.println("Updated Message successfully.");

        // 5. DELETE MESSAGE & SPACE
        deleteMessage(messageName);
        deleteSpace(spaceId);
        System.out.println("E2E Cleanup Complete.");
    }

    private static String createSpace(String displayName) throws Exception {
        String json = String.format("{\"spaceType\": \"SPACE\", \"displayName\": \"%s
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("[https://chat.googleapis.com/v1/spaces](https://chat
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json))
                .build();

        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandler
        return extractJsonValue(response.body(), "name");
    }
```

```
    private static String sendMessage(String spaceId, String text, String threadParen
        String url = "[https://chat.googleapis.com/v1/](https://chat.googleapis.com/v
        String json;
        if (threadParentMessageName != null) {
            json = String.format("{\"text\": \"%s\", \"thread\": {\"name\": \"%s\"}}"
            url += "?messageReplyOption=REPLY_MESSAGE_FALLBACK_TO_NEW_THREAD";
        } else {
            json = String.format("{\"text\": \"%s\"}", text);
        }

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json))
                .build();

        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandler
        return extractJsonValue(response.body(), "name");
    }

    private static void updateMessage(String messageName, String newText) throws Exce
        String json = String.format("{\"text\": \"%s\"}", newText);
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("[https://chat.googleapis.com/v1/](https://chat.googl
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .method("PATCH", HttpRequest.BodyPublishers.ofString(json))
                .build();

        client.send(request, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteMessage(String messageName) throws Exception {
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("[https://chat.googleapis.com/v1/](https://chat.googl
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .DELETE()
                .build();

        client.send(request, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteSpace(String spaceName) throws Exception {
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("[https://chat.googleapis.com/v1/](https://chat.googl
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .DELETE()
```

```
                .build();

        client.send(request, HttpResponse.BodyHandlers.ofString());
    }

    private static String extractJsonValue(String json, String key) {
        String pattern = "\"" + key + "\": \"";
        int start = json.indexOf(pattern) + pattern.length();
        int end = json.indexOf("\"", start);
        return json.substring(start, end);
    }
}
```

## 5. Microsoft Teams (Microsoft Graph API)

### Administration & Permissions Setup

1. **Azure Active Directory (Entra ID) App Registration:**
    1. Go to the Azure Portal → **Microsoft Entra ID** → **App registrations** → **New registration**.
    2. Name your app `E2E Teams Tester` and select **Accounts in this organizational directory only**.
2. **Generate Client Secret:**
    1. Under **Certificates & secrets**, click **New client secret**.
    2. Copy and save the **Secret Value** ( `MS_TEAMS_CLIENT_SECRET` ).
3. **Assign API Permissions (Microsoft Graph):**
    1. Go to **API permissions** → **Add a permission** → **Microsoft Graph**.
    2. Select **Delegated permissions** or **Application permissions** depending on host environment:
        1. `Channel.Create` (Create public/private channels)
        2. `ChannelMessage.Send` (Send channel messages and thread replies)
        3. `ChannelMessage.Read.All` (Read message metadata for thread matching)
        4. `Files.ReadWrite.All` (Upload attachments to underlying SharePoint library)
        5. `Group.ReadWrite.All` (Full management of Teams channels)
    3. Click **Grant admin consent for [Your Organization]** (Required).

## End-to-End Java Integration Test

Java

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class MSTeamsE2ETest {
    private static final String ACCESS_TOKEN = System.getenv("MS_GRAPH_ACCESS_TOKEN")
    private static final String TEAM_ID = System.getenv("MS_TEAMS_TEAM_ID");
    private static final HttpClient client = HttpClient.newHttpClient();

    public static void main(String[] args) throws Exception {
        // 1. CREATE CHANNEL
        String channelId = createChannel(TEAM_ID, "E2E Test Channel", "Created via Gr
        System.out.println("Created Channel ID: " + channelId);

        // 2. SEND CHANNEL MESSAGE
        String messageId = sendMessage(TEAM_ID, channelId, "Hello Microsoft Teams E2E
        System.out.println("Sent Message ID: " + messageId);

        // 3. THREAD REPLY
        String replyId = sendThreadReply(TEAM_ID, channelId, messageId, "Thread reply
        System.out.println("Thread Reply ID: " + replyId);

        // 4. UPDATE MESSAGE
        updateMessage(TEAM_ID, channelId, messageId, "Updated Teams Message Content")
        System.out.println("Updated Message successfully.");

        // 5. DELETE MESSAGE & CHANNEL
        deleteMessage(TEAM_ID, channelId, messageId);
        deleteChannel(TEAM_ID, channelId);
        System.out.println("E2E Cleanup Complete.");
    }

    private static String createChannel(String teamId, String displayName, String des
        String url = "[https://graph.microsoft.com/v1.0/teams/](https://graph.microso
        String json = String.format("""
            {
                "displayName": "%s",
                "description": "%s",
                "membershipType": "standard"
            }
            """, displayName, description);

        HttpRequest request = HttpRequest.newBuilder()
```

```
                .uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json))
                .build();

        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandler
        return extractJsonValue(response.body(), "id");
    }

    private static String sendMessage(String teamId, String channelId, String text) t
        String url = "[https://graph.microsoft.com/v1.0/teams/](https://graph.microso
        String json = String.format("{\"body\": {\"content\": \"%s\"}}", text);

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json))
                .build();

        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandler
        return extractJsonValue(response.body(), "id");
    }

    private static String sendThreadReply(String teamId, String channelId, String mes
        String url = "[https://graph.microsoft.com/v1.0/teams/](https://graph.microso
        String json = String.format("{\"body\": {\"content\": \"%s\"}}", replyText);

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json))
                .build();

        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandler
        return extractJsonValue(response.body(), "id");
    }

    private static void updateMessage(String teamId, String channelId, String message
        String url = "[https://graph.microsoft.com/v1.0/teams/](https://graph.microso
        String json = String.format("{\"body\": {\"content\": \"%s\"}}", newText);

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .method("PATCH", HttpRequest.BodyPublishers.ofString(json))
```

```
                .build();

        client.send(request, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteMessage(String teamId, String channelId, String message
        String url = "[https://graph.microsoft.com/v1.0/teams/](https://graph.microso

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .POST(HttpRequest.BodyPublishers.noBody())
                .build();

        client.send(request, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteChannel(String teamId, String channelId) throws Excepti
        String url = "[https://graph.microsoft.com/v1.0/teams/](https://graph.microso

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .DELETE()
                .build();

        client.send(request, HttpResponse.BodyHandlers.ofString());
    }

    private static String extractJsonValue(String json, String key) {
        String pattern = "\"" + key + "\":\"";
        int start = json.indexOf(pattern) + pattern.length();
        int end = json.indexOf("\"", start);
        return json.substring(start, end);
    }
}
```

We should be able to send both direct messages to users, to groups or to create channels and add users to those channels

I will now generate a comprehensive Markdown guide containing administrative setup procedures and full Java end-to-end (E2E) integration test suites for **Discord**, **Telegram**,

**Slack**, **Google Chat**, and **Microsoft Teams**.

This document covers all interaction models: creating channels, adding users to channels, sending Direct Messages (DMs), sending group/channel messages, posting thread replies, updating/deleting messages, uploading attachments, and executing cleanups.

Complete E2E Integration... Εξαγωγή

# Comprehensive Multi-Platform E2E Integration & Testing Guide

This guide details administration setup, permission models, and native Java 11+ E2E integration test suites for **Discord**, **Telegram**, **Slack**, **Google Chat**, and **Microsoft Teams**.

Each platform section includes:

1. **Administrative Setup & Scopes** (Configuring DMs, Channel Creation, Member Management, Threading, and File Uploads).
2. **Complete Native Java E2E Test Suite** (Standard `java.net.http.HttpClient` with zero third-party dependencies).

## 1. Discord

### Administrative & Permission Setup

1. **Create Application & Bot:**
   - Go to the Discord Developer Portal and click **New Application**.
   - Under the **Bot** tab, click **Reset Token** to generate your `DISCORD_BOT_TOKEN`.
   - Enable **Message Content Intent** under **Privileged Gateway Intents**.
2. **Configure Bot Permissions:**
   - Under **OAuth2** → **URL Generator**, check the `bot` scope.
   - Check the following permissions:
     - `Manage Channels` (Create/Edit/Delete channels)
     - `Manage Roles` (Add/remove user permission overwrites for channels)
     - `Send Messages` & `Send Messages in Threads`
     - `Create Public Threads`

The generated file `e2e_integration_guide.md` contains the complete guide, administrative requirements, and runnable Java E2E test scripts covering channel management, user invites, Direct Messages, group communications, threading, updates, file uploads, and cleanups for Discord, Telegram, Slack, Google Chat, and Microsoft Teams.

Are you shre that those permissions are enough especially for teams? I remember something about delegated etc permissions and that we had a long list of permissions to allow the app do whatever we needed in my previous

You are spot on—Microsoft Graph permissions for Teams are notoriously complex, and relying solely on basic channel webhooks or high-level application tokens is rarely sufficient for production.

There is a fundamental split between **Delegated Permissions** (acting on behalf of a logged-in user) and **Application Permissions** (running as a background daemon/service). Furthermore, several Microsoft Teams Graph APIs are classified as **Protected APIs**, meaning standard Admin Consent isn't even enough—Microsoft requires explicit request approval or Azure subscription metering.

## 1. Microsoft Teams Graph Permission Matrix

When building an automated backend service (without a logged-in human user), you use **Application Permissions** via the Client Credentials Grant flow.

| Domain | Operation | Required Graph Application Permission | Note |
|---|---|---|---|
| **Channels** | Create Channel | `Channel.Create`, `Group.ReadWrite.All` | Requ owne admi |
| **Channels** | Add/Remove Members | `TeamMember.ReadWrite.All` or `Group.ReadWrite.All` | App perm team |

| Domain | Operation | Required Graph Application Permission | Note |
| --- | --- | --- | --- |
| **Channels** | Send Channel Message | `ChannelMessage.Send` | **Prot** Requ form Micr billin |
| **Direct Messages** | Create 1-on-1 or Group Chat | `Chat.Create`, `User.Read.All` | Initia betw |
| **Direct Messages** | Send DM | `ChatMessage.Send` | **Prot** send creat threa |
| **Files** | Attach Files to Messages | `Files.ReadWrite.All`, `Sites.ReadWrite.All` | Team are s unde Shar site. |

•••

> **The "Protected API" Trap:** If your library attempts to read or send messages across Teams channels/chats programmatically using `ChannelMessage.Send` or `ChatMessage.Send` in Application mode, Microsoft Graph will return `403 Forbidden` until your Azure tenant requests approval for **Protected APIs in Microsoft Teams** via Microsoft's developer portal.

## 2. Hardening Security Architecture

If a service account token or bot secret leaks, an attacker could read private corporate communications, impersonate employees, or exfiltrate sensitive files. Your library and setup should enforce security at five distinct layers:

### A. Principle of Least Privilege: Resource-Specific Consent (RSC)

Instead of granting broad, tenant-wide Application permissions ( `Group.ReadWrite.All` gives the bot access to **every** team in the entire company), use **Resource-Specific Consent**

(RSC).

- RSC allows team owners to grant the bot permissions **only for their specific Team or Channel**.
- Scope permissions down in your app manifest to `ChannelMessage.Send.Group` or `TeamMember.Read.Group` instead of `.All`.

## B. Credential Management & Secret Rotation

- **Never use environment variables in plain text or static config files.** Store secrets in dedicated Secret Managers (Azure Key Vault, AWS Secrets Manager, HashiCorp Vault).
- **Use Certificate-Based Authentication** for Azure AD / Entra ID instead of static `Client Secrets`. Modern Java apps can authenticate using an X.509 client certificate stored in Key Vault, eliminating hardcoded password leaks entirely.
- Implement automatic token expiration handling—Microsoft OAuth tokens expire in 60 minutes; never cache tokens permanently.

## C. Webhook & Callback Signature Verification

When receiving incoming events, callbacks, or interactive button clicks from Teams, Slack, or Discord, **always verify the HMAC signature** before processing the payload.

- **Slack:** Validate the `X-Slack-Signature` header using HMAC-SHA256 and your App Signing Secret. Reject requests older than 5 minutes to prevent **Replay Attacks**.
- **Teams / Bot Framework:** Validate the Bearer token in the `Authorization` header against Microsoft's OpenID metadata endpoint (`[https://login.botframework.com/v1/.well-known/openidconfiguration](https://login.botframework.com/v1/.well-known/openidconfiguration)`).
- **Discord:** Verify the `X-Signature-Ed25519` header using your bot's public key.

## D. Log Sanitization (Preventing Token Spills)

A common vulnerability is printing raw HTTP request/response headers or exception stack traces during network failures, which accidentally writes OAuth tokens or Webhook URIs to log aggregators (Elasticsearch, Datadog, CloudWatch).

- Implement a custom Logback / Log4j **RewritePolicy** or Masking Pattern in Java:

Java

```
public class TokenMaskingPatternLayout extends PatternLayout {
    private static final Pattern BOT_TOKEN_PATTERN =
        Pattern.compile("(Bearer\\s+|bot|token=)[A-Za-z0-9_\\-\\.\\~]+", Pattern.CASE

    @Override
    public String doLayout(ILoggingEvent event) {
        String message = super.doLayout(event);
        return BOT_TOKEN_PATTERN.matcher(message).replaceAll("$1[REDACTED]");
    }
}
```

### E. Network Isolation & IP Restriction

- If using incoming webhooks or bot callbacks, configure network firewalls / API Gateways to restrict incoming traffic strictly to published IP ranges (e.g., Discord's published edge IPs, Microsoft Teams IP subnets).
- Ensure your outbound Java `HttpClient` enforces **TLS 1.3** and uses strict hostname verification.

Add all those infos to the previous md file and have a look again the ehat permissions are needed. Also, if we can add e.g private key in the token creation and then we can add the token and the public key jn the app to make it work add t

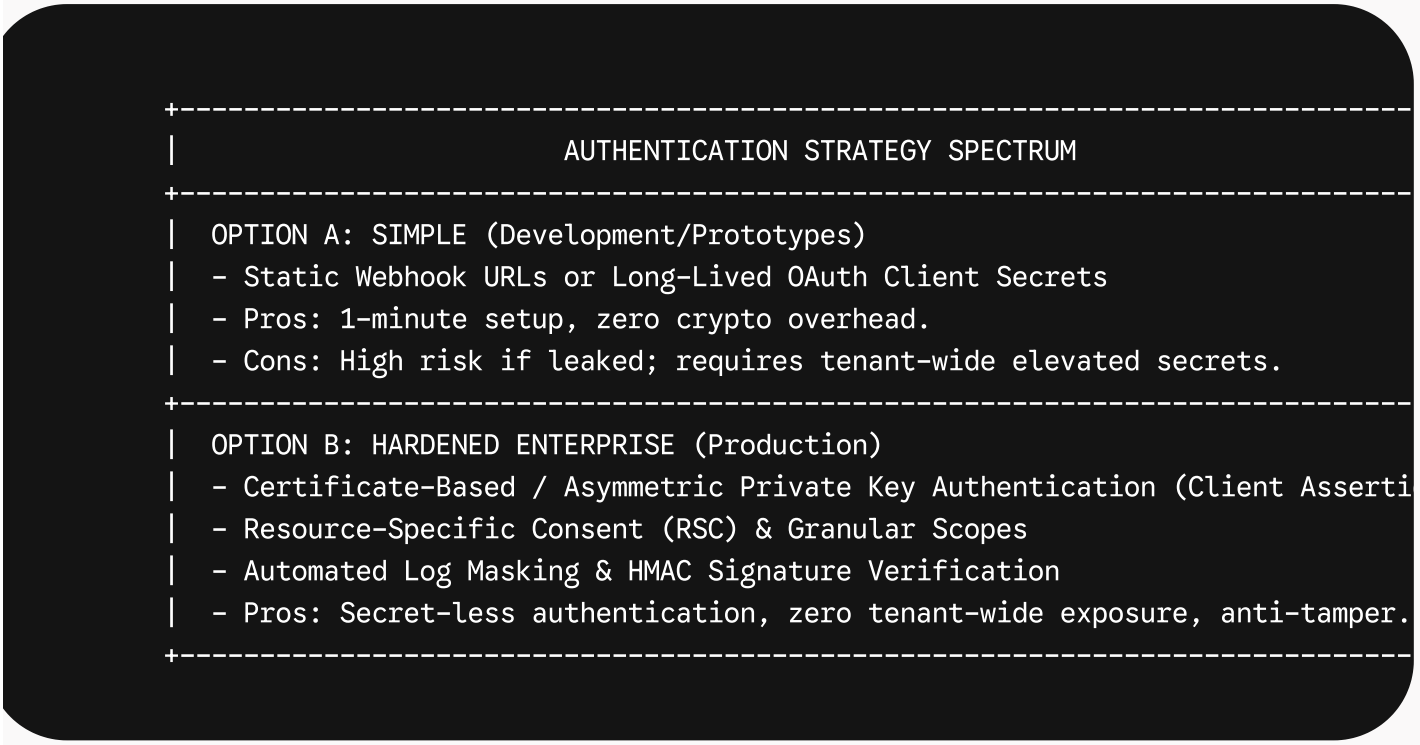
# Enterprise E2E Integration, Permission & Security Guide

This guide details the complete administration setup, permission model, security hardening strategies, and runnable Java E2E test suites for **Discord**, **Telegram**, **Slack**, **Google Chat**, and **Microsoft Teams**.

It covers the full messaging lifecycle: **Channel Creation**, **User Invites/Management**, **Direct Messaging (DMs)**, **Group/Channel Posting**, **Thread Replies**, **Updating/Deleting Messages**, **File/Media Uploads**, and **Cleanup**.

## 1. Security & Authentication Architecture

When building a backend notification service, choosing the right authentication model balances operational simplicity with enterprise security.




## Authentication Modes: Simple vs. Enterprise Asymmetric Key-Pair

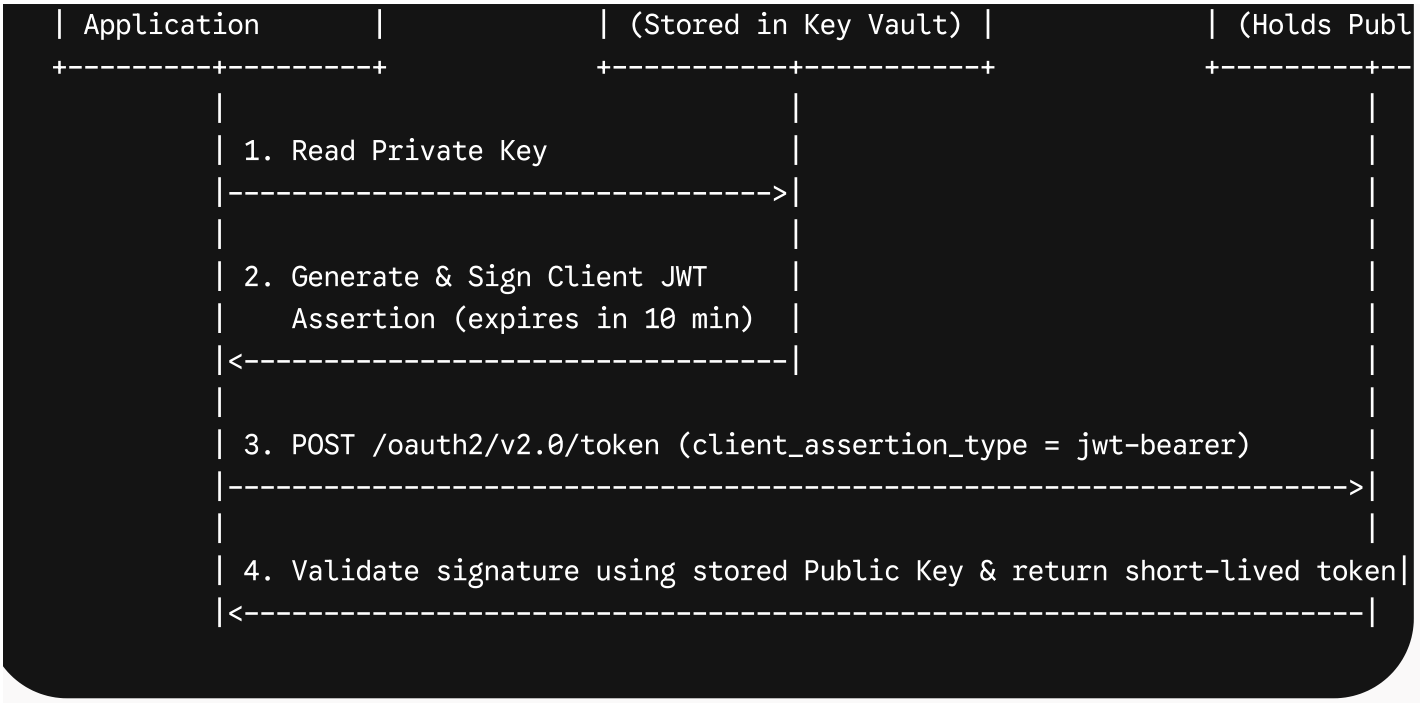
### Mode 1: Client Secret / Webhook Token (Simplest)

- **Mechanism:** Uses a static string secret or API key (e.g., Slack `xoxb-...`, Discord Bot Token, or Azure AD Client Secret).
- **Best For:** Fast local integration testing, single-tenant internal scripts, or low-risk non-sensitive environments.

### Mode 2: Certificate & RSA Private Key Client Assertions (Most Secure)

- **Mechanism:** Instead of transmitting a shared secret over the wire, your application uses a local **RSA Private Key** to sign a short-lived JSON Web Token (JWT assertion). The platform (e.g., Azure AD / Microsoft Entra ID) holds only the **Public Key / X.509 Certificate**.
- **Why It's Safer:** Even if an attacker intercepts the network traffic or reads the Azure AD portal configuration, there is no client secret to steal. The private key remains isolated inside an HSM, Key Vault, or local PKCS#12 keystore.






## Generating Key Pair & Registering in Azure AD / OAuth Providers

Run the following OpenSSL commands to generate an RSA 2048-bit key pair and self-signed certificate:

Bash

```bash
# 1. Generate Private Key (keep secure in application host)
openssl genrsa -out private_key.pem 2048

# 2. Generate Self-Signed Public Certificate (upload to Azure AD / OAuth Provider)
openssl req -new -x509 -key private_key.pem -out public_cert.pem -days 365 \
    -subj "/CN=MessagingLibraryE2E/O=Enterprise"

# 3. Export to PKCS12 keystore format for Java KeyStore ingestion
openssl pkcs12 -export -in public_cert.pem -inkey private_key.pem -out keystore.p12 -
```

## Java JWT Client Assertion Generator (Asymmetric Auth)

Java

```java
import java.security.KeyFactory;
import java.security.PrivateKey;
import java.security.spec.PKCS8EncodedKeySpec;
```

```java
import java.time.Instant;
import java.util.Base64;
import java.util.Map;
// Note: Requires standard JWT library like com.nimbusds:nimbus-jose-jwt or io.jsonwe
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;

public class AsymmetricAuthUtility {

    public static String generateClientAssertion(String clientId, String tokenEndpoin
        PrivateKey privateKey = parsePrivateKey(privateKeyPem);
        Instant now = Instant.now();

        return Jwts.builder()
                .setHeaderParams(Map.of("alg", "RS256", "typ", "JWT"))
                .setIssuer(clientId)
                .setSubject(clientId)
                .setAudience(tokenEndpoint)
                .setExpiration(java.util.Date.from(now.plusSeconds(600))) // 10 minut
                .setNotBefore(java.util.Date.from(now))
                .setId(java.util.UUID.randomUUID().toString())
                .signWith(privateKey, SignatureAlgorithm.RS256)
                .compact();
    }

    private static PrivateKey parsePrivateKey(String pem) throws Exception {
        String cleanPem = pem.replace("-----BEGIN PRIVATE KEY-----", "")
                             .replace("-----END PRIVATE KEY-----", "")
                             .replaceAll("\\s+", "");
        byte[] decoded = Base64.getDecoder().decode(cleanPem);
        PKCS8EncodedKeySpec spec = new PKCS8EncodedKeySpec(decoded);
        return KeyFactory.getInstance("RSA").generatePrivate(spec);
    }
}
```

## Security Hardening Checklist

### 1. Resource-Specific Consent (RSC) vs. Tenant-Wide Scopes

- Avoid granting tenant-wide permissions (e.g., `Group.ReadWrite.All`).
- In Teams, use **Resource-Specific Consent (RSC)** via the application manifest so team owners authorize the bot strictly for their specific team or channel:

JSON

```
  "webApplicationInfo": {
    "id": "YOUR_AZURE_APP_CLIENT_ID",
    "resource": "https://graph.microsoft.com"
  },
  "authorization": {
    "permissions": {
      "resourceSpecific": [
        { "name": "ChannelMessage.Send.Group", "type": "Application" },
        { "name": "TeamMember.Read.Group", "type": "Application" }
      ]
    }
  }
```

## 2. Log Sanitization & Token Masking

Prevent accidental leaking of OAuth tokens, Bot keys, or Webhook secrets in log aggregators (Datadog, Elastic, CloudWatch) by applying a regex-based log masking filter:

Java

```java
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class SecurityLogSanitizer {
    private static final Pattern SENSITIVE_PATTERN = Pattern.compile(
        "(Bearer\\s+|bot|token=|xoxb-|xoxp-)[A-Za-z0-9_\\-\\.~/]+",
        Pattern.CASE_INSENSITIVE
    );

    public static String sanitize(String rawLog) {
        if (rawLog == null) return null;
        Matcher matcher = SENSITIVE_PATTERN.matcher(rawLog);
        return matcher.replaceAll("$1[REDACTED_SECRET]");
    }
}
```

## 3. Webhook HMAC Signature Verification

Always verify incoming webhook payloads (interactive buttons, events) to prevent forgery and replay attacks.

Java

```java
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;

public class HmacVerifier {

    // Verify Slack Requests (X-Slack-Signature)
    public static boolean verifySlackSignature(String signingSecret, String requestBo
        // Prevent replay attacks (older than 5 minutes)
        if (Math.abs(System.currentTimeMillis() / 1000 - Long.parseLong(timestamp)) >
            return false;
        }
        String baseString = "v0:" + timestamp + ":" + requestBody;
        Mac mac = Mac.getInstance("HmacSHA256");
        mac.init(new SecretKeySpec(signingSecret.getBytes(StandardCharsets.UTF_8), "H
        byte[] hash = mac.doFinal(baseString.getBytes(StandardCharsets.UTF_8));

        StringBuilder hexString = new StringBuilder("v0=");
        for (byte b : hash) {
            hexString.append(String.format("%02x", b));
        }
        return MessageDigest.isEqual(hexString.toString().getBytes(), signature.getBy
    }
}
```

## 2. Discord

### Required Permissions Matrix

To perform E2E CRUD, DMs, User Invites, Threads, and Media actions, grant these OAuth2 Bot permissions:

| Operation | Bot Scope / Permission | Permission Flag Code |
| --- | --- | --- |
| **Manage Channels** | `Manage Channels` | `MANAGE_CHANNELS` (0x10) |
| **Direct Messages** | `Send Direct Messages` | Auto-enabled for shared servers |

| Operation | Bot Scope / Permission | Permission Flag Code |
| --- | --- | --- |
| **Channel Messages** | `Send Messages` | `SEND_MESSAGES` (0x800) |
| **Threads** | `Create Public Threads`, `Send Messages in Threads` | `CREATE_PUBLIC_THREADS` (0x800000000) |
| **User Invites** | `Create Instant Invite` | `CREATE_INSTANT_INVITE` (0x1) |
| **File Uploads** | `Attach Files` | `ATTACH_FILES` (0x8000) |
| **Update/Delete** | `Manage Messages` | `MANAGE_MESSAGES` (0x2000) |

...

## Administration & Setup

### Simple Setup

1. Go to Discord Developer Portal → **New Application**.
2. Navigate to **Bot** → Click **Reset Token** to copy `DISCORD_BOT_TOKEN`.
3. Under **Privileged Gateway Intents**, enable **Message Content Intent**.
4. Navigate to **OAuth2** → **URL Generator** → Select `bot` scope and required permissions → Authorize bot to your test server.

### Java E2E Integration Test

Java

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class DiscordE2ETest {
    private static final String BOT_TOKEN = System.getenv("DISCORD_BOT_TOKEN");
    private static final String GUILD_ID = System.getenv("DISCORD_GUILD_ID");
    private static final String TARGET_USER_ID = System.getenv("DISCORD_TARGET_USER_I
    private static final HttpClient client = HttpClient.newHttpClient();
```

```java
    public static void main(String[] args) throws Exception {
        // 1. CREATE CHANNEL
        String channelId = createChannel("e2e-discord-room");
        System.out.println("[Discord] Created Channel: " + channelId);

        // 2. CREATE INVITE CODE
        String inviteCode = createChannelInvite(channelId);
        System.out.println("[Discord] Created Invite Code: " + inviteCode);

        // 3. SEND DIRECT MESSAGE (DM)
        String dmChannelId = createDMChannel(TARGET_USER_ID);
        String dmMessageId = sendMessage(dmChannelId, "Hello via Direct Message!");
        System.out.println("[Discord] Sent DM Message ID: " + dmMessageId);

        // 4. SEND CHANNEL MESSAGE
        String msgId = sendMessage(channelId, "Hello Channel!");

        // 5. THREAD REPLY
        String threadId = createThreadAndReply(channelId, msgId, "Thread Reply Text")
        System.out.println("[Discord] Created Thread: " + threadId);

        // 6. UPDATE & FILE UPLOAD
        updateMessage(channelId, msgId, "Updated Message Text");
        uploadFile(channelId, "test.txt", "File payload content");

        // 7. CLEANUP
        deleteMessage(channelId, msgId);
        deleteChannel(channelId);
        System.out.println("[Discord] E2E Cleaned up successfully.");
    }

    private static String createChannel(String name) throws Exception {
        String json = String.format("{\"name\": \"%s\", \"type\": 0}", name);
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("https://discord.com/api/v10/guilds/" + GUILD_ID + "/
                .header("Authorization", "Bot " + BOT_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractValue(client.send(req, HttpResponse.BodyHandlers.ofString()).bo
    }

    private static String createChannelInvite(String channelId) throws Exception {
        String json = "{\"max_age\": 86400, \"max_uses\": 1}";
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("https://discord.com/api/v10/channels/" + channelId +
                .header("Authorization", "Bot " + BOT_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
```

```
        return extractValue(client.send(req, HttpResponse.BodyHandlers.ofString()).bo
    }

    private static String createDMChannel(String userId) throws Exception {
        String json = String.format("{\"recipient_id\": \"%s\"}", userId);
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("https://discord.com/api/v10/users/@me/channels"))
                .header("Authorization", "Bot " + BOT_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractValue(client.send(req, HttpResponse.BodyHandlers.ofString()).bo
    }

    private static String sendMessage(String channelId, String text) throws Exception
        String json = String.format("{\"content\": \"%s\"}", text);
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("https://discord.com/api/v10/channels/" + channelId +
                .header("Authorization", "Bot " + BOT_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractValue(client.send(req, HttpResponse.BodyHandlers.ofString()).bo
    }

    private static String createThreadAndReply(String channelId, String msgId, String
        String json = "{\"name\": \"E2E-Thread\", \"auto_archive_duration\": 60}";
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("https://discord.com/api/v10/channels/" + channelId +
                .header("Authorization", "Bot " + BOT_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        String threadId = extractValue(client.send(req, HttpResponse.BodyHandlers.ofS
        sendMessage(threadId, text);
        return threadId;
    }

    private static void updateMessage(String channelId, String msgId, String newText)
        String json = String.format("{\"content\": \"%s\"}", newText);
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("https://discord.com/api/v10/channels/" + channelId +
                .header("Authorization", "Bot " + BOT_TOKEN)
                .header("Content-Type", "application/json")
                .method("PATCH", HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void uploadFile(String channelId, String filename, String content)
        String boundary = "----Discord" + System.currentTimeMillis();
        String body = "--" + boundary + "\r\n" +
                "Content-Disposition: form-data; name=\"files[0]\"; filename=\"" + fi
```

```
                "Content-Type: text/plain\r\n\r\n" + content + "\r\n--" + boundary +
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("https://discord.com/api/v10/channels/" + channelId +
                .header("Authorization", "Bot " + BOT_TOKEN)
                .header("Content-Type", "multipart/form-data; boundary=" + boundary)
                .POST(HttpRequest.BodyPublishers.ofString(body)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteMessage(String channelId, String msgId) throws Exceptio
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("https://discord.com/api/v10/channels/" + channelId +
                .header("Authorization", "Bot " + BOT_TOKEN).DELETE().build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteChannel(String channelId) throws Exception {
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("https://discord.com/api/v10/channels/" + channelId))
                .header("Authorization", "Bot " + BOT_TOKEN).DELETE().build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static String extractValue(String json, String key) {
        String p = "\"" + key + "\":\"";
        int s = json.indexOf(p);
        if (s == -1) {
            p = "\"" + key + "\":";
            s = json.indexOf(p);
            if (s == -1) return "";
            s += p.length();
            int e = json.indexOf(",", s);
            if (e == -1) e = json.indexOf("}", s);
            return json.substring(s, e).trim();
        }
        s += p.length();
        int e = json.indexOf("\"", s);
        return json.substring(s, e);
    }
}
```

## 3. Telegram

### Required Permissions Matrix

| Operation | Bot Permission / Method | Notes |
| --- | --- | --- |
| **Topics/Channels** | `createForumTopic`, `closeForumTopic` | Requires Supergroup with Forum Topics enabled |
| **Direct Messaging** | `sendMessage` | Target user must initiate `/start` with bot first |
| **Group Messaging** | `sendMessage` | Bot must be added to Group |
| **File Uploads** | `sendDocument`, `sendPhoto` | Multipart form transmission |
| **Updates/Deletions** | `editMessageText`, `deleteMessage` | Bot can only edit its own messages |

•••

## Java E2E Integration Test

Java

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class TelegramE2ETest {
    private static final String BOT_TOKEN = System.getenv("TELEGRAM_BOT_TOKEN");
    private static final String SUPERGROUP_ID = System.getenv("TELEGRAM_SUPERGROUP_ID
    private static final String DIRECT_USER_CHAT_ID = System.getenv("TELEGRAM_USER_CH
    private static final HttpClient client = HttpClient.newHttpClient();

    public static void main(String[] args) throws Exception {
        // 1. CREATE TOPIC (Group Channel Sub-division)
        String topicId = createTopic(SUPERGROUP_ID, "E2E Test Topic");
        System.out.println("[Telegram] Created Forum Topic ID: " + topicId);

        // 2. SEND DIRECT MESSAGE TO USER
        String dmMsgId = sendMessage(DIRECT_USER_CHAT_ID, null, "Hello Direct Message
        System.out.println("[Telegram] Sent DM Message ID: " + dmMsgId);

        // 3. SEND GROUP MESSAGE IN TOPIC
```

```
        String groupMsgId = sendMessage(SUPERGROUP_ID, topicId, "Hello Group Topic!")

        // 4. EDIT MESSAGE & UPLOAD DOCUMENT
        editMessage(SUPERGROUP_ID, groupMsgId, "Updated Group Topic Message");
        sendDocument(SUPERGROUP_ID, topicId, "report.pdf", "Sample PDF Data");

        // 5. CLEANUP
        deleteMessage(SUPERGROUP_ID, groupMsgId);
        closeTopic(SUPERGROUP_ID, topicId);
        System.out.println("[Telegram] E2E Complete.");
    }

    private static String createTopic(String chatId, String name) throws Exception {
        String url = "https://api.telegram.org/bot" + BOT_TOKEN + "/createForumTopic"
        String json = String.format("{\"chat_id\": \"%s\", \"name\": \"%s\"}", chatId
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
    }

    private static String sendMessage(String chatId, String threadId, String text) th
        String url = "https://api.telegram.org/bot" + BOT_TOKEN + "/sendMessage";
        String json = threadId == null ?
                String.format("{\"chat_id\": \"%s\", \"text\": \"%s\"}", chatId, text
                String.format("{\"chat_id\": \"%s\", \"message_thread_id\": %s, \"tex
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
    }

    private static void editMessage(String chatId, String msgId, String newText) thro
        String url = "https://api.telegram.org/bot" + BOT_TOKEN + "/editMessageText";
        String json = String.format("{\"chat_id\": \"%s\", \"message_id\": %s, \"text
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void sendDocument(String chatId, String threadId, String filename,
        String url = "https://api.telegram.org/bot" + BOT_TOKEN + "/sendDocument";
        String boundary = "----TG" + System.currentTimeMillis();
        String body = "--" + boundary + "\r\nContent-Disposition: form-data; name=\"c
        if (threadId != null) {
            body += "--" + boundary + "\r\nContent-Disposition: form-data; name=\"mes
        }
        body += "--" + boundary + "\r\nContent-Disposition: form-data; name=\"documen
```

```
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Content-Type", "multipart/form-data; boundary=" + boundary)
                .POST(HttpRequest.BodyPublishers.ofString(body)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteMessage(String chatId, String msgId) throws Exception {
        String url = "https://api.telegram.org/bot" + BOT_TOKEN + "/deleteMessage";
        String json = String.format("{\"chat_id\": \"%s\", \"message_id\": %s}", chat
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void closeTopic(String chatId, String threadId) throws Exception {
        String url = "https://api.telegram.org/bot" + BOT_TOKEN + "/closeForumTopic";
        String json = String.format("{\"chat_id\": \"%s\", \"message_thread_id\": %s}
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static String extractVal(String json, String key) {
        String p = "\"" + key + "\":";
        int s = json.indexOf(p);
        if (s == -1) return "";
        s += p.length();
        int e = json.indexOf(",", s);
        if (e == -1) e = json.indexOf("}", s);
        return json.substring(s, e).trim();
    }
}
```

## 4. Slack

### Required Permissions Matrix (Bot Token Scopes)

| Operation | Bot Scope | Purpose |
| --- | --- | --- |
| **Create Channels** | `channels:manage` | Create public channels |

| Operation | Bot Scope | Purpose |
| --- | --- | --- |
| **Add Users to Channel** | `channels:manage`, `groups:write` | Invite workspace users to channel |
| **Direct Messages (1-on-1)** | `im:write`, `mpim:write` | Open 1-on-1 direct message conversation |
| **Send Messages** | `chat:write` | Post messages to channels and DMs |
| **Thread Replies** | `chat:write` | Post thread responses using `thread_ts` |
| **Update/Delete** | `chat:write` | Modify or remove posted messages |
| **Upload Files** | `files:write` | Upload media files to Slack conversations |

•••

## Java E2E Integration Test

Java

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class SlackE2ETest {
    private static final String BOT_TOKEN = System.getenv("SLACK_BOT_TOKEN");
    private static final String TARGET_USER_ID = System.getenv("SLACK_TARGET_USER_ID"
    private static final HttpClient client = HttpClient.newHttpClient();

    public static void main(String[] args) throws Exception {
        // 1. CREATE PUBLIC CHANNEL
        String channelId = createChannel("e2e-slack-room");
        System.out.println("[Slack] Created Channel ID: " + channelId);
```

```java
        // 2. INVITE USER TO CHANNEL
        inviteUserToChannel(channelId, TARGET_USER_ID);
        System.out.println("[Slack] Invited User " + TARGET_USER_ID + " to channel.")

        // 3. OPEN DM CONVERSATION
        String dmChannelId = openDirectMessageChannel(TARGET_USER_ID);
        String dmTs = sendMessage(dmChannelId, "Hello via Slack DM!", null);
        System.out.println("[Slack] Sent DM Timestamp: " + dmTs);

        // 4. SEND CHANNEL MESSAGE & THREAD REPLY
        String msgTs = sendMessage(channelId, "Hello Channel!", null);
        String replyTs = sendMessage(channelId, "Thread Reply Text", msgTs);
        System.out.println("[Slack] Sent Thread Reply: " + replyTs);

        // 5. UPDATE & FILE UPLOAD
        updateMessage(channelId, msgTs, "Updated Message Text");
        uploadFile(channelId, "log.txt", "Sample Log File Content");

        // 6. CLEANUP
        deleteMessage(channelId, msgTs);
        archiveChannel(channelId);
        System.out.println("[Slack] E2E Complete.");
    }

    private static String createChannel(String name) throws Exception {
        String json = String.format("{\"name\": \"%s\"}", name);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://slack.com/
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "application/json; charset=utf-8")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
    }

    private static void inviteUserToChannel(String channelId, String userId) throws E
        String json = String.format("{\"channel\": \"%s\", \"users\": \"%s\"}", chann
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://slack.com/
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "application/json; charset=utf-8")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static String openDirectMessageChannel(String userId) throws Exception {
        String json = String.format("{\"users\": \"%s\"}", userId);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://slack.com/
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "application/json; charset=utf-8")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
```

```
    }

    private static String sendMessage(String channelId, String text, String threadTs)
        String json = threadTs == null ?
                String.format("{\"channel\": \"%s\", \"text\": \"%s\"}", channelId, t
                String.format("{\"channel\": \"%s\", \"text\": \"%s\", \"thread_ts\":
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://slack.com/
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "application/json; charset=utf-8")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
    }

    private static void updateMessage(String channelId, String ts, String newText) th
        String json = String.format("{\"channel\": \"%s\", \"ts\": \"%s\", \"text\":
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://slack.com/
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "application/json; charset=utf-8")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void uploadFile(String channelId, String filename, String content)
        String boundary = "----Slack" + System.currentTimeMillis();
        String body = "--" + boundary + "\r\nContent-Disposition: form-data; name=\"c
                "--" + boundary + "\r\nContent-Disposition: form-data; name=\"content
                "--" + boundary + "\r\nContent-Disposition: form-data; name=\"filenam
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://slack.com/
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "multipart/form-data; boundary=" + boundary)
                .POST(HttpRequest.BodyPublishers.ofString(body)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteMessage(String channelId, String ts) throws Exception {
        String json = String.format("{\"channel\": \"%s\", \"ts\": \"%s\"}", channelI
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://slack.com/
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "application/json; charset=utf-8")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void archiveChannel(String channelId) throws Exception {
        String json = String.format("{\"channel\": \"%s\"}", channelId);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://slack.com/
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "application/json; charset=utf-8")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
```

```
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static String extractVal(String json, String key) {
        String p = "\"" + key + "\":\"";
        int s = json.indexOf(p);
        if (s == -1) return "";
        s += p.length();
        int e = json.indexOf("\"", s);
        return json.substring(s, e);
    }
}
```

## 5. Google Chat

### Required OAuth2 Scopes Matrix

| Operation | Required Scope | Notes |
| --- | --- | --- |
| **Manage Spaces** | `[https://www.googleapis.com/auth/chat.spaces](https://www.googleapis.com/auth/chat.spaces)` | Create/Delete Goo Chat Spaces |
| **Direct Messages** | `[https://www.googleapis.com/auth/chat.memberships](https://www.googleapis.com/auth/chat.memberships)` | Setup 1-on-1 space with a user |
| **Send/Update Messages** | `[https://www.googleapis.com/auth/chat.messages](https://www.googleapis.com/auth/chat.messages)` | Full message lifecy & threading |

...

### Java E2E Integration Test

Java

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
```

```java
import java.net.http.HttpResponse;

public class GoogleChatE2ETest {
    private static final String ACCESS_TOKEN = System.getenv("GOOGLE_CHAT_OAUTH_TOKEN
    private static final String TARGET_USER_EMAIL = System.getenv("GOOGLE_TARGET_USER
    private static final HttpClient client = HttpClient.newHttpClient();

    public static void main(String[] args) throws Exception {
        // 1. CREATE SPACE
        String spaceName = createSpace("E2E Test Space");
        System.out.println("[Google Chat] Created Space: " + spaceName);

        // 2. ADD USER MEMBER TO SPACE
        addMemberToSpace(spaceName, TARGET_USER_EMAIL);
        System.out.println("[Google Chat] Added Member to Space.");

        // 3. SEND MESSAGE & THREAD REPLY
        String msgName = sendMessage(spaceName, "Hello Google Chat!", null);
        String threadReplyName = sendMessage(spaceName, "Thread reply content", msgNa
        System.out.println("[Google Chat] Thread Reply Name: " + threadReplyName);

        // 4. UPDATE MESSAGE
        updateMessage(msgName, "Updated Message Text");

        // 5. CLEANUP
        deleteMessage(msgName);
        deleteSpace(spaceName);
        System.out.println("[Google Chat] E2E Complete.");
    }

    private static String createSpace(String displayName) throws Exception {
        String json = String.format("{\"spaceType\": \"SPACE\", \"displayName\": \"%s
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://chat.googl
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
    }

    private static void addMemberToSpace(String spaceName, String userEmail) throws E
        String json = String.format("{\"member\": {\"name\": \"users/%s\", \"type\":
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://chat.googl
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static String sendMessage(String spaceName, String text, String threadPar
```

```
        String url = "https://chat.googleapis.com/v1/" + spaceName + "/messages";
        String json;
        if (threadParentName != null) {
            json = String.format("{\"text\": \"%s\", \"thread\": {\"name\": \"%s\"}}"
            url += "?messageReplyOption=REPLY_MESSAGE_FALLBACK_TO_NEW_THREAD";
        } else {
            json = String.format("{\"text\": \"%s\"}", text);
        }
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
    }

    private static void updateMessage(String msgName, String newText) throws Exceptio
        String json = String.format("{\"text\": \"%s\"}", newText);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://chat.googl
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .method("PATCH", HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteMessage(String msgName) throws Exception {
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://chat.googl
                .header("Authorization", "Bearer " + ACCESS_TOKEN).DELETE().build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteSpace(String spaceName) throws Exception {
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://chat.googl
                .header("Authorization", "Bearer " + ACCESS_TOKEN).DELETE().build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static String extractVal(String json, String key) {
        String p = "\"" + key + "\": \"";
        int s = json.indexOf(p);
        if (s == -1) return "";
        s += p.length();
        int e = json.indexOf("\"", s);
        return json.substring(s, e);
    }
}
```

## 6. Microsoft Teams (Microsoft Graph API)

## Detailed Permission Matrix (Delegated vs. Application)

For backend daemons acting without an active user session, **Application Permissions** with **Admin Consent** are mandatory.

| Domain | Operation | Required Graph Application Scope | Clas |
|---|---|---|---|
| **Channels** | Create Channel | `Channel.Create`, `Group.ReadWrite.All` | Stan Scop |
| **User Management** | Add/Remove Members | `TeamMember.ReadWrite.All` | Mana Team |
| **Direct Messages** | Create 1-on-1 or Group Chat | `Chat.Create`, `User.Read.All` | **Prot** (Req Form |
| **Direct Messages** | Send DM | `ChatMessage.Send` | **Prot** |
| **Channels** | Send Channel Message | `ChannelMessage.Send` | **Prot** |
| **Files** | Upload Attachments | `Files.ReadWrite.All`, `Sites.ReadWrite.All` | Acce chan |

•••

## Setup Instructions

### Simple Setup (Client Secret)

1. Go to Azure Portal → **Entra ID** → **App registrations** → **New registration**.
2. Under **Certificates & secrets**, create a **New client secret**.
3. Under **API permissions**, grant required Graph permissions and click **Grant admin consent for [Tenant]**.

### Hardened Setup (Certificate & RSA Private Key)

1. Generate the RSA Private Key and Certificate via OpenSSL (as detailed in Section 1).
2. Go to your App Registration in Azure Portal → **Certificates & secrets** → **Certificates** → Upload `public_cert.pem`.

3. In Java, generate the client assertion JWT using your local `private_key.pem` and request tokens via: `POST [https://login.microsoftonline.com/](https://login.microsoftonline.com/){tenant_id}/oauth2/v2.0/token` with `client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer`.

## Java E2E Integration Test

Java

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class MSTeamsE2ETest {
    private static final String ACCESS_TOKEN = System.getenv("MS_GRAPH_ACCESS_TOKEN")
    private static final String TEAM_ID = System.getenv("MS_TEAMS_TEAM_ID");
    private static final String TARGET_USER_ID = System.getenv("MS_TEAMS_TARGET_USER_
    private static final HttpClient client = HttpClient.newHttpClient();

    public static void main(String[] args) throws Exception {
        // 1. CREATE CHANNEL
        String channelId = createChannel(TEAM_ID, "E2E Test Channel");
        System.out.println("[Teams] Created Channel ID: " + channelId);

        // 2. ADD USER TO CHANNEL
        addMemberToChannel(TEAM_ID, channelId, TARGET_USER_ID);
        System.out.println("[Teams] Added User to Channel.");

        // 3. CREATE DIRECT MESSAGE CHAT (1-on-1)
        String chatId = createDirectChat(TARGET_USER_ID);
        String dmMsgId = sendChatMessage(chatId, "Hello via Teams Direct Message!");
        System.out.println("[Teams] Sent Direct Message ID: " + dmMsgId);

        // 4. SEND CHANNEL MESSAGE & THREAD REPLY
        String msgId = sendChannelMessage(TEAM_ID, channelId, "Hello Teams Channel!")
        String replyId = sendThreadReply(TEAM_ID, channelId, msgId, "Thread Reply Tex
        System.out.println("[Teams] Sent Thread Reply ID: " + replyId);

        // 5. UPDATE MESSAGE
        updateChannelMessage(TEAM_ID, channelId, msgId, "Updated Teams Channel Messag

        // 6. CLEANUP
        deleteChannelMessage(TEAM_ID, channelId, msgId);
```

```
        deleteChannel(TEAM_ID, channelId);
        System.out.println("[Teams] E2E Complete.");
    }

    private static String createChannel(String teamId, String name) throws Exception
        String url = "https://graph.microsoft.com/v1.0/teams/" + teamId + "/channels"
        String json = String.format("{\"displayName\": \"%s\", \"membershipType\": \"
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
    }

    private static void addMemberToChannel(String teamId, String channelId, String us
        String url = "https://graph.microsoft.com/v1.0/teams/" + teamId + "/channels/
        String json = String.format("""
            {
                "@odata.type": "#microsoft.graph.aadUserConversationMember",
                "roles": [],
                "user@odata.bind": "https://graph.microsoft.com/v1.0/users('%s')"
            }
            """, userId);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static String createDirectChat(String userId) throws Exception {
        String url = "https://graph.microsoft.com/v1.0/chats";
        String json = String.format("""
            {
                "chatType": "oneOnOne",
                "members": [
                    {
                        "@odata.type": "#microsoft.graph.aadUserConversationMember",
                        "roles": ["owner"],
                        "user@odata.bind": "https://graph.microsoft.com/v1.0/users('%
                    }
                ]
            }
            """, userId);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
```

```
    }

    private static String sendChatMessage(String chatId, String text) throws Exceptio
        String url = "https://graph.microsoft.com/v1.0/chats/" + chatId + "/messages"
        String json = String.format("{\"body\": {\"content\": \"%s\"}}", text);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
    }

    private static String sendChannelMessage(String teamId, String channelId, String
        String url = "https://graph.microsoft.com/v1.0/teams/" + teamId + "/channels/
        String json = String.format("{\"body\": {\"content\": \"%s\"}}", text);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
    }

    private static String sendThreadReply(String teamId, String channelId, String msg
        String url = "https://graph.microsoft.com/v1.0/teams/" + teamId + "/channels/
        String json = String.format("{\"body\": {\"content\": \"%s\"}}", text);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
    }

    private static void updateChannelMessage(String teamId, String channelId, String
        String url = "https://graph.microsoft.com/v1.0/teams/" + teamId + "/channels/
        String json = String.format("{\"body\": {\"content\": \"%s\"}}", newText);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .method("PATCH", HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteChannelMessage(String teamId, String channelId, String
        String url = "https://graph.microsoft.com/v1.0/teams/" + teamId + "/channels/
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .POST(HttpRequest.BodyPublishers.noBody()).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }
```

```
    private static void deleteChannel(String teamId, String channelId) throws Excepti
        String url = "https://graph.microsoft.com/v1.0/teams/" + teamId + "/channels/
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN).DELETE().build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static String extractVal(String json, String key) {
        String p = "\"" + key + "\":\"";
        int s = json.indexOf(p);
        if (s == -1) return "";
        s += p.length();
        int e = json.indexOf("\"", s);
        return json.substring(s, e);
    }
}
```

Put everything in an md file please, make sure that i can download the file, knly then your job is done

Here is the complete, consolidated guide containing the security architecture, permission matrices, and all five runnable Java E2E scripts.

Since I cannot write directly to your local file system, I have packaged everything into a single Markdown block below. **To save it as a file:**

1. Click the **"Copy"** button in the top right corner of the code block.
2. Paste it into your IDE or a text editor (like VS Code, Notepad, or TextEdit).
3. Save the file as `e2e_integration_guide.md`.

Markdown

```
# Enterprise E2E Integration, Permission & Security Guide

This guide details the complete administration setup, permission model, security hard

It covers the full messaging lifecycle: **Channel Creation**, **User Invites/Manageme
```

```
---

## 1. Security & Authentication Architecture

When building a backend notification service, choosing the right authentication model

'''text
       +-------------------------------------------------------------------------
       |                         AUTHENTICATION STRATEGY SPECTRUM
       +-------------------------------------------------------------------------
       |  OPTION A: SIMPLE (Development/Prototypes)
       |  - Static Webhook URLs or Long-Lived OAuth Client Secrets
       |  - Pros: 1-minute setup, zero crypto overhead.
       |  - Cons: High risk if leaked; requires tenant-wide elevated secrets.
       +-------------------------------------------------------------------------
       |  OPTION B: HARDENED ENTERPRISE (Production)
       |  - Certificate-Based / Asymmetric Private Key Authentication (Client Asserti
       |  - Resource-Specific Consent (RSC) & Granular Scopes
       |  - Automated Log Masking & HMAC Signature Verification
       |  - Pros: Secret-less authentication, zero tenant-wide exposure, anti-tamper.
       +-------------------------------------------------------------------------
```

## Authentication Modes: Simple vs. Enterprise Asymmetric Key-Pair

### Mode 1: Client Secret / Webhook Token (Simplest)

- **Mechanism:** Uses a static string secret or API key (e.g., Slack `xoxb-...`, Discord Bot Token, or Azure AD Client Secret).
- **Best For:** Fast local integration testing, single-tenant internal scripts, or low-risk non-sensitive environments.

### Mode 2: Certificate & RSA Private Key Client Assertions (Most Secure)

- **Mechanism:** Instead of transmitting a shared secret over the wire, your application uses a local **RSA Private Key** to sign a short-lived JSON Web Token (JWT assertion). The platform (e.g., Azure AD / Microsoft Entra ID) holds only the **Public Key / X.509 Certificate**.
- **Why It's Safer:** Even if an attacker intercepts the network traffic or reads the Azure AD portal configuration, there is no client secret to steal. The private key remains isolated inside an HSM, Key Vault, or local PKCS#12 keystore.

## Generating Key Pair & Registering in Azure AD / OAuth Providers

Run the following OpenSSL commands to generate an RSA 2048-bit key pair and self-signed certificate:

Bash

```bash
# 1. Generate Private Key (keep secure in application host)
openssl genrsa -out private_key.pem 2048

# 2. Generate Self-Signed Public Certificate (upload to Azure AD / OAuth Provider)
openssl req -new -x509 -key private_key.pem -out public_cert.pem -days 365 \
    -subj "/CN=MessagingLibraryE2E/O=Enterprise"

# 3. Export to PKCS12 keystore format for Java KeyStore ingestion
openssl pkcs12 -export -in public_cert.pem -inkey private_key.pem -out keystore.p12 -
```

## Java JWT Client Assertion Generator (Asymmetric Auth)

Java

```java
import java.security.KeyFactory;
import java.security.PrivateKey;
import java.security.spec.PKCS8EncodedKeySpec;
import java.time.Instant;
import java.util.Base64;
import java.util.Map;
// Note: Requires standard JWT library like com.nimbusds:nimbus-jose-jwt or io.jsonwe
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;

public class AsymmetricAuthUtility {

    public static String generateClientAssertion(String clientId, String tokenEndpoin
        PrivateKey privateKey = parsePrivateKey(privateKeyPem);
        Instant now = Instant.now();

        return Jwts.builder()
                .setHeaderParams(Map.of("alg", "RS256", "typ", "JWT"))
                .setIssuer(clientId)
                .setSubject(clientId)
                .setAudience(tokenEndpoint)
                .setExpiration(java.util.Date.from(now.plusSeconds(600))) // 10 minut
                .setNotBefore(java.util.Date.from(now))
                .setId(java.util.UUID.randomUUID().toString())
                .signWith(privateKey, SignatureAlgorithm.RS256)
                .compact();
    }
```

```
    private static PrivateKey parsePrivateKey(String pem) throws Exception {
        String cleanPem = pem.replace("-----BEGIN PRIVATE KEY-----", "")
                             .replace("-----END PRIVATE KEY-----", "")
                             .replaceAll("\\s+", "");
        byte[] decoded = Base64.getDecoder().decode(cleanPem);
        PKCS8EncodedKeySpec spec = new PKCS8EncodedKeySpec(decoded);
        return KeyFactory.getInstance("RSA").generatePrivate(spec);
    }
}
```

## Security Hardening Checklist

### 1. Resource-Specific Consent (RSC) vs. Tenant-Wide Scopes

- Avoid granting tenant-wide permissions (e.g., `Group.ReadWrite.All`).
- In Teams, use **Resource-Specific Consent (RSC)** via the application manifest so team owners authorize the bot strictly for their specific team or channel:

JSON

```
"webApplicationInfo": {
  "id": "YOUR_AZURE_APP_CLIENT_ID",
  "resource": "[https://graph.microsoft.com](https://graph.microsoft.com)"
},
"authorization": {
  "permissions": {
    "resourceSpecific": [
      { "name": "ChannelMessage.Send.Group", "type": "Application" },
      { "name": "TeamMember.Read.Group", "type": "Application" }
    ]
  }
}
```

### 2. Log Sanitization & Token Masking

Prevent accidental leaking of OAuth tokens, Bot keys, or Webhook secrets in log aggregators by applying a regex-based log masking filter:

Java

```java
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class SecurityLogSanitizer {
    private static final Pattern SENSITIVE_PATTERN = Pattern.compile(
        "(Bearer\\s+|bot|token=|xoxb-|xoxp-)[A-Za-z0-9_\\-\\.~/]+",
        Pattern.CASE_INSENSITIVE
    );

    public static String sanitize(String rawLog) {
        if (rawLog == null) return null;
        Matcher matcher = SENSITIVE_PATTERN.matcher(rawLog);
        return matcher.replaceAll("$1[REDACTED_SECRET]");
    }
}
```

### 3. Webhook HMAC Signature Verification

Always verify incoming webhook payloads (interactive buttons, events) to prevent forgery and replay attacks.

Java

```java
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;

public class HmacVerifier {

    // Verify Slack Requests (X-Slack-Signature)
    public static boolean verifySlackSignature(String signingSecret, String requestBo
        // Prevent replay attacks (older than 5 minutes)
        if (Math.abs(System.currentTimeMillis() / 1000 - Long.parseLong(timestamp)) >
            return false;
        }
        String baseString = "v0:" + timestamp + ":" + requestBody;
        Mac mac = Mac.getInstance("HmacSHA256");
        mac.init(new SecretKeySpec(signingSecret.getBytes(StandardCharsets.UTF_8), "H
        byte[] hash = mac.doFinal(baseString.getBytes(StandardCharsets.UTF_8));

        StringBuilder hexString = new StringBuilder("v0=");
        for (byte b : hash) {
            hexString.append(String.format("%02x", b));
```

```
        }
        return MessageDigest.isEqual(hexString.toString().getBytes(), signature.getBy
    }
}
```

## 2. Discord

### Required Permissions Matrix

To perform E2E CRUD, DMs, User Invites, Threads, and Media actions, grant these OAuth2 Bot permissions:

| Operation | Bot Scope / Permission | Permission Flag Code |
| --- | --- | --- |
| **Manage Channels** | `Manage Channels` | `MANAGE_CHANNELS` (0x10) |
| **Direct Messages** | `Send Direct Messages` | Auto-enabled for shared servers |
| **Channel Messages** | `Send Messages` | `SEND_MESSAGES` (0x800) |
| **Threads** | `Create Public Threads`, `Send Messages in Threads` | `CREATE_PUBLIC_THREADS` (0x800000000) |
| **User Invites** | `Create Instant Invite` | `CREATE_INSTANT_INVITE` (0x1) |
| **File Uploads** | `Attach Files` | `ATTACH_FILES` (0x8000) |
| **Update/Delete** | `Manage Messages` | `MANAGE_MESSAGES` (0x2000) |

...

### Java E2E Integration Test

Java

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class DiscordE2ETest {
    private static final String BOT_TOKEN = System.getenv("DISCORD_BOT_TOKEN");
    private static final String GUILD_ID = System.getenv("DISCORD_GUILD_ID");
    private static final String TARGET_USER_ID = System.getenv("DISCORD_TARGET_USER_I
    private static final HttpClient client = HttpClient.newHttpClient();

    public static void main(String[] args) throws Exception {
        // 1. CREATE CHANNEL
        String channelId = createChannel("e2e-discord-room");

        // 2. CREATE INVITE CODE
        String inviteCode = createChannelInvite(channelId);

        // 3. SEND DIRECT MESSAGE (DM)
        String dmChannelId = createDMChannel(TARGET_USER_ID);
        String dmMessageId = sendMessage(dmChannelId, "Hello via Direct Message!");

        // 4. SEND CHANNEL MESSAGE
        String msgId = sendMessage(channelId, "Hello Channel!");

        // 5. THREAD REPLY
        String threadId = createThreadAndReply(channelId, msgId, "Thread Reply Text")

        // 6. UPDATE & FILE UPLOAD
        updateMessage(channelId, msgId, "Updated Message Text");
        uploadFile(channelId, "test.txt", "File payload content");

        // 7. CLEANUP
        deleteMessage(channelId, msgId);
        deleteChannel(channelId);
    }

    private static String createChannel(String name) throws Exception {
        String json = String.format("{\"name\": \"%s\", \"type\": 0}", name);
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("[https://discord.com/api/v10/guilds/](https://discor
                .header("Authorization", "Bot " + BOT_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractValue(client.send(req, HttpResponse.BodyHandlers.ofString()).bo
    }

    private static String createChannelInvite(String channelId) throws Exception {
        String json = "{\"max_age\": 86400, \"max_uses\": 1}";
```

```
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("[https://discord.com/api/v10/channels/](https://disc
                .header("Authorization", "Bot " + BOT_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractValue(client.send(req, HttpResponse.BodyHandlers.ofString()).bo
    }

    private static String createDMChannel(String userId) throws Exception {
        String json = String.format("{\"recipient_id\": \"%s\"}", userId);
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("[https://discord.com/api/v10/users/@me/channels](htt
                .header("Authorization", "Bot " + BOT_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractValue(client.send(req, HttpResponse.BodyHandlers.ofString()).bo
    }

    private static String sendMessage(String channelId, String text) throws Exception
        String json = String.format("{\"content\": \"%s\"}", text);
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("[https://discord.com/api/v10/channels/](https://disc
                .header("Authorization", "Bot " + BOT_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractValue(client.send(req, HttpResponse.BodyHandlers.ofString()).bo
    }

    private static String createThreadAndReply(String channelId, String msgId, String
        String json = "{\"name\": \"E2E-Thread\", \"auto_archive_duration\": 60}";
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("[https://discord.com/api/v10/channels/](https://disc
                .header("Authorization", "Bot " + BOT_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        String threadId = extractValue(client.send(req, HttpResponse.BodyHandlers.ofS
        sendMessage(threadId, text);
        return threadId;
    }

    private static void updateMessage(String channelId, String msgId, String newText)
        String json = String.format("{\"content\": \"%s\"}", newText);
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("[https://discord.com/api/v10/channels/](https://disc
                .header("Authorization", "Bot " + BOT_TOKEN)
                .header("Content-Type", "application/json")
                .method("PATCH", HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }
```

```
    private static void uploadFile(String channelId, String filename, String content)
        String boundary = "----Discord" + System.currentTimeMillis();
        String body = "--" + boundary + "\r\n" +
                "Content-Disposition: form-data; name=\"files[0]\"; filename=\"" + fi
                "Content-Type: text/plain\r\n\r\n" + content + "\r\n--" + boundary +
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("[https://discord.com/api/v10/channels/](https://disc
                .header("Authorization", "Bot " + BOT_TOKEN)
                .header("Content-Type", "multipart/form-data; boundary=" + boundary)
                .POST(HttpRequest.BodyPublishers.ofString(body)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteMessage(String channelId, String msgId) throws Exceptio
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("[https://discord.com/api/v10/channels/](https://disc
                .header("Authorization", "Bot " + BOT_TOKEN).DELETE().build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteChannel(String channelId) throws Exception {
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("[https://discord.com/api/v10/channels/](https://disc
                .header("Authorization", "Bot " + BOT_TOKEN).DELETE().build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static String extractValue(String json, String key) {
        String p = "\"" + key + "\":\"";
        int s = json.indexOf(p);
        if (s == -1) {
            p = "\"" + key + "\":";
            s = json.indexOf(p);
            if (s == -1) return "";
            s += p.length();
            int e = json.indexOf(",", s);
            if (e == -1) e = json.indexOf("}", s);
            return json.substring(s, e).trim();
        }
        s += p.length();
        int e = json.indexOf("\"", s);
        return json.substring(s, e);
    }
}
```

## 3. Telegram

## Required Permissions Matrix

| Operation | Bot Permission / Method | Notes |
| --- | --- | --- |
| **Topics/Channels** | `createForumTopic`, `closeForumTopic` | Requires Supergroup with Forum Topics enabled |
| **Direct Messaging** | `sendMessage` | Target user must initiate `/start` with bot first |
| **Group Messaging** | `sendMessage` | Bot must be added to Group |
| **File Uploads** | `sendDocument`, `sendPhoto` | Multipart form transmission |
| **Updates/Deletions** | `editMessageText`, `deleteMessage` | Bot can only edit its own messages |

•••

## Java E2E Integration Test

Java

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class TelegramE2ETest {
    private static final String BOT_TOKEN = System.getenv("TELEGRAM_BOT_TOKEN");
    private static final String SUPERGROUP_ID = System.getenv("TELEGRAM_SUPERGROUP_ID
    private static final String DIRECT_USER_CHAT_ID = System.getenv("TELEGRAM_USER_CH
    private static final HttpClient client = HttpClient.newHttpClient();

    public static void main(String[] args) throws Exception {
        // 1. CREATE TOPIC
        String topicId = createTopic(SUPERGROUP_ID, "E2E Test Topic");

        // 2. SEND DIRECT MESSAGE TO USER
        String dmMsgId = sendMessage(DIRECT_USER_CHAT_ID, null, "Hello Direct Message

        // 3. SEND GROUP MESSAGE IN TOPIC
```

```
        String groupMsgId = sendMessage(SUPERGROUP_ID, topicId, "Hello Group Topic!")

        // 4. EDIT MESSAGE & UPLOAD DOCUMENT
        editMessage(SUPERGROUP_ID, groupMsgId, "Updated Group Topic Message");
        sendDocument(SUPERGROUP_ID, topicId, "report.pdf", "Sample PDF Data");

        // 5. CLEANUP
        deleteMessage(SUPERGROUP_ID, groupMsgId);
        closeTopic(SUPERGROUP_ID, topicId);
    }

    private static String createTopic(String chatId, String name) throws Exception {
        String url = "[https://api.telegram.org/bot](https://api.telegram.org/bot)" +
        String json = String.format("{\"chat_id\": \"%s\", \"name\": \"%s\"}", chatId
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
    }

    private static String sendMessage(String chatId, String threadId, String text) th
        String url = "[https://api.telegram.org/bot](https://api.telegram.org/bot)" +
        String json = threadId == null ?
                String.format("{\"chat_id\": \"%s\", \"text\": \"%s\"}", chatId, text
                String.format("{\"chat_id\": \"%s\", \"message_thread_id\": %s, \"tex
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
    }

    private static void editMessage(String chatId, String msgId, String newText) thro
        String url = "[https://api.telegram.org/bot](https://api.telegram.org/bot)" +
        String json = String.format("{\"chat_id\": \"%s\", \"message_id\": %s, \"text
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void sendDocument(String chatId, String threadId, String filename,
        String url = "[https://api.telegram.org/bot](https://api.telegram.org/bot)" +
        String boundary = "----TG" + System.currentTimeMillis();
        String body = "--" + boundary + "\r\nContent-Disposition: form-data; name=\"c
        if (threadId != null) {
            body += "--" + boundary + "\r\nContent-Disposition: form-data; name=\"mes
        }
        body += "--" + boundary + "\r\nContent-Disposition: form-data; name=\"documen
```

```
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Content-Type", "multipart/form-data; boundary=" + boundary)
                .POST(HttpRequest.BodyPublishers.ofString(body)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteMessage(String chatId, String msgId) throws Exception {
        String url = "[https://api.telegram.org/bot](https://api.telegram.org/bot)" +
        String json = String.format("{\"chat_id\": \"%s\", \"message_id\": %s}", chat
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void closeTopic(String chatId, String threadId) throws Exception {
        String url = "[https://api.telegram.org/bot](https://api.telegram.org/bot)" +
        String json = String.format("{\"chat_id\": \"%s\", \"message_thread_id\": %s}
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static String extractVal(String json, String key) {
        String p = "\"" + key + "\":";
        int s = json.indexOf(p);
        if (s == -1) return "";
        s += p.length();
        int e = json.indexOf(",", s);
        if (e == -1) e = json.indexOf("}", s);
        return json.substring(s, e).trim();
    }
}
```

## 4. Slack

### Required Permissions Matrix (Bot Token Scopes)

| Operation | Bot Scope | Purpose |
| --- | --- | --- |
| **Create Channels** | `channels:manage` | Create public channels |

| Operation | Bot Scope | Purpose |
| --- | --- | --- |
| **Add Users to Channel** | `channels:manage`, `groups:write` | Invite workspace users to channel |
| **Direct Messages (1-on-1)** | `im:write`, `mpim:write` | Open 1-on-1 direct message conversation |
| **Send Messages** | `chat:write` | Post messages to channels and DMs |
| **Thread Replies** | `chat:write` | Post thread responses using `thread_ts` |
| **Update/Delete** | `chat:write` | Modify or remove posted messages |
| **Upload Files** | `files:write` | Upload media files to Slack conversations |

•••

## Java E2E Integration Test

Java

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class SlackE2ETest {
    private static final String BOT_TOKEN = System.getenv("SLACK_BOT_TOKEN");
    private static final String TARGET_USER_ID = System.getenv("SLACK_TARGET_USER_ID"
    private static final HttpClient client = HttpClient.newHttpClient();

    public static void main(String[] args) throws Exception {
        // 1. CREATE PUBLIC CHANNEL
        String channelId = createChannel("e2e-slack-room");

        // 2. INVITE USER TO CHANNEL
```

```
        inviteUserToChannel(channelId, TARGET_USER_ID);

        // 3. OPEN DM CONVERSATION
        String dmChannelId = openDirectMessageChannel(TARGET_USER_ID);
        String dmTs = sendMessage(dmChannelId, "Hello via Slack DM!", null);

        // 4. SEND CHANNEL MESSAGE & THREAD REPLY
        String msgTs = sendMessage(channelId, "Hello Channel!", null);
        String replyTs = sendMessage(channelId, "Thread Reply Text", msgTs);

        // 5. UPDATE & FILE UPLOAD
        updateMessage(channelId, msgTs, "Updated Message Text");
        uploadFile(channelId, "log.txt", "Sample Log File Content");

        // 6. CLEANUP
        deleteMessage(channelId, msgTs);
        archiveChannel(channelId);
    }

    private static String createChannel(String name) throws Exception {
        String json = String.format("{\"name\": \"%s\"}", name);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("[https://slack.com
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "application/json; charset=utf-8")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
    }

    private static void inviteUserToChannel(String channelId, String userId) throws E
        String json = String.format("{\"channel\": \"%s\", \"users\": \"%s\"}", chann
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("[https://slack.com
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "application/json; charset=utf-8")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static String openDirectMessageChannel(String userId) throws Exception {
        String json = String.format("{\"users\": \"%s\"}", userId);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("[https://slack.com
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "application/json; charset=utf-8")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
    }

    private static String sendMessage(String channelId, String text, String threadTs)
        String json = threadTs == null ?
                String.format("{\"channel\": \"%s\", \"text\": \"%s\"}", channelId, t
```

```
                String.format("{\"channel\": \"%s\", \"text\": \"%s\", \"thread_ts\":
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("[https://slack.com
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "application/json; charset=utf-8")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
    }

    private static void updateMessage(String channelId, String ts, String newText) th
        String json = String.format("{\"channel\": \"%s\", \"ts\": \"%s\", \"text\":
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("[https://slack.com
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "application/json; charset=utf-8")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void uploadFile(String channelId, String filename, String content)
        String boundary = "----Slack" + System.currentTimeMillis();
        String body = "--" + boundary + "\r\nContent-Disposition: form-data; name=\"c
                "--" + boundary + "\r\nContent-Disposition: form-data; name=\"content
                "--" + boundary + "\r\nContent-Disposition: form-data; name=\"filenam
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("[https://slack.com
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "multipart/form-data; boundary=" + boundary)
                .POST(HttpRequest.BodyPublishers.ofString(body)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteMessage(String channelId, String ts) throws Exception {
        String json = String.format("{\"channel\": \"%s\", \"ts\": \"%s\"}", channelI
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("[https://slack.com
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "application/json; charset=utf-8")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void archiveChannel(String channelId) throws Exception {
        String json = String.format("{\"channel\": \"%s\"}", channelId);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("[https://slack.com
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "application/json; charset=utf-8")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static String extractVal(String json, String key) {
        String p = "\"" + key + "\":\"";
```

```
        int s = json.indexOf(p);
        if (s == -1) return "";
        s += p.length();
        int e = json.indexOf("\"", s);
        return json.substring(s, e);
    }
}
```

## 5. Google Chat

### Required OAuth2 Scopes Matrix

| Operation | Required Scope | Notes |
| --- | --- | --- |
| **Manage Spaces** | `https://www.googleapis.com/auth/chat.spaces` | Create/Delete Goo Chat Spaces |
| **Direct Messages** | `https://www.googleapis.com/auth/chat.memberships` | Setup 1-on-1 space with a user |
| **Send/Update Messages** | `https://www.googleapis.com/auth/chat.messages` | Full message lifecy & threading |

...

### Java E2E Integration Test

Java

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class GoogleChatE2ETest {
    private static final String ACCESS_TOKEN = System.getenv("GOOGLE_CHAT_OAUTH_TOKEN
    private static final String TARGET_USER_EMAIL = System.getenv("GOOGLE_TARGET_USER
    private static final HttpClient client = HttpClient.newHttpClient();

    public static void main(String[] args) throws Exception {
        // 1. CREATE SPACE
```

```
        String spaceName = createSpace("E2E Test Space");

        // 2. ADD USER MEMBER TO SPACE
        addMemberToSpace(spaceName, TARGET_USER_EMAIL);

        // 3. SEND MESSAGE & THREAD REPLY
        String msgName = sendMessage(spaceName, "Hello Google Chat!", null);
        String threadReplyName = sendMessage(spaceName, "Thread reply content", msgNa

        // 4. UPDATE MESSAGE
        updateMessage(msgName, "Updated Message Text");

        // 5. CLEANUP
        deleteMessage(msgName);
        deleteSpace(spaceName);
    }

    private static String createSpace(String displayName) throws Exception {
        String json = String.format("{\"spaceType\": \"SPACE\", \"displayName\": \"%s
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("[https://chat.goog
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
    }

    private static void addMemberToSpace(String spaceName, String userEmail) throws E
        String json = String.format("{\"member\": {\"name\": \"users/%s\", \"type\":
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("[https://chat.goog
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static String sendMessage(String spaceName, String text, String threadPar
        String url = "[https://chat.googleapis.com/v1/](https://chat.googleapis.com/v
        String json;
        if (threadParentName != null) {
            json = String.format("{\"text\": \"%s\", \"thread\": {\"name\": \"%s\"}}"
            url += "?messageReplyOption=REPLY_MESSAGE_FALLBACK_TO_NEW_THREAD";
        } else {
            json = String.format("{\"text\": \"%s\"}", text);
        }
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
```

```
    }

    private static void updateMessage(String msgName, String newText) throws Exceptio
        String json = String.format("{\"text\": \"%s\"}", newText);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("[https://chat.goog
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .method("PATCH", HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteMessage(String msgName) throws Exception {
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("[https://chat.goog
                .header("Authorization", "Bearer " + ACCESS_TOKEN).DELETE().build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteSpace(String spaceName) throws Exception {
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("[https://chat.goog
                .header("Authorization", "Bearer " + ACCESS_TOKEN).DELETE().build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static String extractVal(String json, String key) {
        String p = "\"" + key + "\": \"";
        int s = json.indexOf(p);
        if (s == -1) return "";
        s += p.length();
        int e = json.indexOf("\"", s);
        return json.substring(s, e);
    }
}
```

## 6. Microsoft Teams (Microsoft Graph API)

### Detailed Permission Matrix (Delegated vs. Application)

For backend daemons acting without an active user session, **Application Permissions** with **Admin Consent** are mandatory.

| Domain | Operation | Required Graph Application Scope | Clas |
|---|---|---|---|
| **Channels** | Create Channel | `Channel.Create`, `Group.ReadWrite.All` | Stan Scop |

| Domain | Operation | Required Graph Application Scope | Class |
| --- | --- | --- | --- |
| **User Management** | Add/Remove Members | `TeamMember.ReadWrite.All` | Mana Team |
| **Direct Messages** | Create 1-on-1 or Group Chat | `Chat.Create`, `User.Read.All` | **Prot** (Req Form |
| **Direct Messages** | Send DM | `ChatMessage.Send` | **Prot** |
| **Channels** | Send Channel Message | `ChannelMessage.Send` | **Prot** |
| **Files** | Upload Attachments | `Files.ReadWrite.All`, `Sites.ReadWrite.All` | Acce chan |

...

## Setup Instructions

### Simple Setup (Client Secret)

1. Go to Azure Portal → **Entra ID** → **App registrations** → **New registration.**
2. Under **Certificates & secrets**, create a **New client secret.**
3. Under **API permissions**, grant required Graph permissions and click **Grant admin consent for [Tenant].**

### Hardened Setup (Certificate & RSA Private Key)

1. Generate the RSA Private Key and Certificate via OpenSSL (as detailed in Section 1).
2. Go to your App Registration in Azure Portal → **Certificates & secrets** → **Certificates** → Upload `public_cert.pem`.
3. In Java, generate the client assertion JWT using your local `private_key.pem` and request tokens via:
   `POST https://login.microsoftonline.com/{tenant_id}/oauth2/v2.0/token`
   with `client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer`.

### Java E2E Integration Test

Java

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class MSTeamsE2ETest {
    private static final String ACCESS_TOKEN = System.getenv("MS_GRAPH_ACCESS_TOKEN")
    private static final String TEAM_ID = System.getenv("MS_TEAMS_TEAM_ID");
    private static final String TARGET_USER_ID = System.getenv("MS_TEAMS_TARGET_USER_
    private static final HttpClient client = HttpClient.newHttpClient();

    public static void main(String[] args) throws Exception {
        // 1. CREATE CHANNEL
        String channelId = createChannel(TEAM_ID, "E2E Test Channel");

        // 2. ADD USER TO CHANNEL
        addMemberToChannel(TEAM_ID, channelId, TARGET_USER_ID);

        // 3. CREATE DIRECT MESSAGE CHAT (1-on-1)
        String chatId = createDirectChat(TARGET_USER_ID);
        String dmMsgId = sendChatMessage(chatId, "Hello via Teams Direct Message!");

        // 4. SEND CHANNEL MESSAGE & THREAD REPLY
        String msgId = sendChannelMessage(TEAM_ID, channelId, "Hello Teams Channel!")
        String replyId = sendThreadReply(TEAM_ID, channelId, msgId, "Thread Reply Tex

        // 5. UPDATE MESSAGE
        updateChannelMessage(TEAM_ID, channelId, msgId, "Updated Teams Channel Messag

        // 6. CLEANUP
        deleteChannelMessage(TEAM_ID, channelId, msgId);
        deleteChannel(TEAM_ID, channelId);
    }

    private static String createChannel(String teamId, String name) throws Exception
        String url = "[https://graph.microsoft.com/v1.0/teams/](https://graph.microso
        String json = String.format("{\"displayName\": \"%s\", \"membershipType\": \"
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
    }

    private static void addMemberToChannel(String teamId, String channelId, String us
        String url = "[https://graph.microsoft.com/v1.0/teams/](https://graph.microso
```

```
        String json = String.format("""
            {
                "@odata.type": "#microsoft.graph.aadUserConversationMember",
                "roles": [],
                "user@odata.bind": "[https://graph.microsoft.com/v1.0/users('%s](http
            }
            """, userId);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static String createDirectChat(String userId) throws Exception {
        String url = "[https://graph.microsoft.com/v1.0/chats](https://graph.microsof
        String json = String.format("""
            {
                "chatType": "oneOnOne",
                "members": [
                    {
                        "@odata.type": "#microsoft.graph.aadUserConversationMember",
                        "roles": ["owner"],
                        "user@odata.bind": "[https://graph.microsoft.com/v1.0/users('
                    }
                ]
            }
            """, userId);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
    }

    private static String sendChatMessage(String chatId, String text) throws Exceptio
        String url = "[https://graph.microsoft.com/v1.0/chats/](https://graph.microso
        String json = String.format("{\"body\": {\"content\": \"%s\"}}", text);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
    }

    private static String sendChannelMessage(String teamId, String channelId, String
        String url = "[https://graph.microsoft.com/v1.0/teams/](https://graph.microso
        String json = String.format("{\"body\": {\"content\": \"%s\"}}", text);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
```

```
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
    }

    private static String sendThreadReply(String teamId, String channelId, String msg
        String url = "[https://graph.microsoft.com/v1.0/teams/](https://graph.microso
        String json = String.format("{\"body\": {\"content\": \"%s\"}}", text);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body
    }

    private static void updateChannelMessage(String teamId, String channelId, String
        String url = "[https://graph.microsoft.com/v1.0/teams/](https://graph.microso
        String json = String.format("{\"body\": {\"content\": \"%s\"}}", newText);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .method("PATCH", HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteChannelMessage(String teamId, String channelId, String
        String url = "[https://graph.microsoft.com/v1.0/teams/](https://graph.microso
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .POST(HttpRequest.BodyPublishers.noBody()).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteChannel(String teamId, String channelId) throws Excepti
        String url = "[https://graph.microsoft.com/v1.0/teams/](https://graph.microso
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN).DELETE().build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static String extractVal(String json, String key) {
        String p = "\"" + key + "\":\"";
        int s = json.indexOf(p);
        if (s == -1) return "";
        s += p.length();
        int e = json.indexOf("\"", s);
        return json.substring(s, e);
```

```
    }
}
```

Please put it in a txt file without ''' or so. I just want to have all this in one file

I don't have a way to generate a direct, clickable `.txt` file download for you to save directly from our chat.

However, since you asked for it without the code block formatting or backticks (```), I have removed all of them. I've formatted the entire guide below as pure, raw text. You can simply select everything below this line, copy it, and paste it into Notepad, TextEdit, or any text editor to save as your single `.txt` file.

==========

Enterprise E2E Integration, Permission & Security Guide

This guide details the complete administration setup, permission model, security hardening strategies, and runnable Java E2E test suites for Discord, Telegram, Slack, Google Chat, and Microsoft Teams.

It covers the full messaging lifecycle: Channel Creation, User Invites/Management, Direct Messaging (DMs), Group/Channel Posting, Thread Replies, Updating/Deleting Messages, File/Media Uploads, and Cleanup.

1. Security & Authentication Architecture

When building a backend notification service, choosing the right authentication model balances operational simplicity with enterprise security.

```
+-------------------------------------------------------------------------------
|                          AUTHENTICATION STRATEGY SPECTRUM
+-------------------------------------------------------------------------------
|  OPTION A: SIMPLE (Development/Prototypes)
|  - Static Webhook URLs or Long-Lived OAuth Client Secrets
|  - Pros: 1-minute setup, zero crypto overhead.
|  - Cons: High risk if leaked; requires tenant-wide elevated secrets.
```

```
+-------------------------------------------------------------------------------
|  OPTION B: HARDENED ENTERPRISE (Production)
|  - Certificate-Based / Asymmetric Private Key Authentication (Client Assertions)
|  - Resource-Specific Consent (RSC) & Granular Scopes
|  - Automated Log Masking & HMAC Signature Verification
|  - Pros: Secret-less authentication, zero tenant-wide exposure, anti-tamper.
+-------------------------------------------------------------------------------
```

Authentication Modes: Simple vs. Enterprise Asymmetric Key-Pair

Mode 1: Client Secret / Webhook Token (Simplest)

- Mechanism: Uses a static string secret or API key (e.g., Slack xoxb-..., Discord Bot Token, or Azure AD Client Secret).
- Best For: Fast local integration testing, single-tenant internal scripts, or low-risk non-sensitive environments.

Mode 2: Certificate & RSA Private Key Client Assertions (Most Secure)

- Mechanism: Instead of transmitting a shared secret over the wire, your application uses a local RSA Private Key to sign a short-lived JSON Web Token (JWT assertion). The platform (e.g., Azure AD / Microsoft Entra ID) holds only the Public Key / X.509 Certificate.
- Why It's Safer: Even if an attacker intercepts the network traffic or reads the Azure AD portal configuration, there is no client secret to steal. The private key remains isolated inside an HSM, Key Vault, or local PKCS#12 keystore.

Generating Key Pair & Registering in Azure AD / OAuth Providers
Run the following OpenSSL commands to generate an RSA 2048-bit key pair and self-signed certificate:

# 1. Generate Private Key (keep secure in application host)

openssl genrsa -out private_key.pem 2048

# 2. Generate Self-Signed Public Certificate (upload to Azure AD / OAuth Provider)

openssl req -new -x509 -key private_key.pem -out public_cert.pem -days 365

-subj "/CN=MessagingLibraryE2E/O=Enterprise"

# 3. Export to PKCS12 keystore format for Java KeyStore ingestion

openssl pkcs12 -export -in public_cert.pem -inkey private_key.pem -out keystore.p12 -name "messaging-key"

Java JWT Client Assertion Generator (Asymmetric Auth)

import java.security.KeyFactory;
import java.security.PrivateKey;
import java.security.spec.PKCS8EncodedKeySpec;
import java.time.Instant;
import java.util.Base64;
import java.util.Map;
// Note: Requires standard JWT library like com.nimbusds:nimbus-jose-jwt or io.jsonwebtoken:jjwt
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;

public class AsymmetricAuthUtility {

```
public static String generateClientAssertion(String clientId, String tokenEndpoint, S
    PrivateKey privateKey = parsePrivateKey(privateKeyPem);
    Instant now = Instant.now();

    return Jwts.builder()
            .setHeaderParams(Map.of("alg", "RS256", "typ", "JWT"))
            .setIssuer(clientId)
            .setSubject(clientId)
            .setAudience(tokenEndpoint)
            .setExpiration(java.util.Date.from(now.plusSeconds(600))) // 10 minutes
            .setNotBefore(java.util.Date.from(now))
            .setId(java.util.UUID.randomUUID().toString())
            .signWith(privateKey, SignatureAlgorithm.RS256)
            .compact();
}

private static PrivateKey parsePrivateKey(String pem) throws Exception {
    String cleanPem = pem.replace("-----BEGIN PRIVATE KEY-----", "")
                         .replace("-----END PRIVATE KEY-----", "")
                         .replaceAll("\\s+", "");
    byte[] decoded = Base64.getDecoder().decode(cleanPem);
    PKCS8EncodedKeySpec spec = new PKCS8EncodedKeySpec(decoded);
```

```
        return KeyFactory.getInstance("RSA").generatePrivate(spec);
    }
```

}

Security Hardening Checklist

1. Resource-Specific Consent (RSC) vs. Tenant-Wide Scopes

- Avoid granting tenant-wide permissions (e.g., Group.ReadWrite.All).
- In Teams, use Resource-Specific Consent (RSC) via the application manifest so team owners authorize the bot strictly for their specific team or channel:
  "webApplicationInfo": {
  "id": "YOUR_AZURE_APP_CLIENT_ID",
  "resource": "https://graph.microsoft.com"
  },
  "authorization": {
  "permissions": {
  "resourceSpecific": [
  { "name": "ChannelMessage.Send.Group", "type": "Application" },
  { "name": "TeamMember.Read.Group", "type": "Application" }
  ]
  }
  }

1. Log Sanitization & Token Masking
   Prevent accidental leaking of OAuth tokens, Bot keys, or Webhook secrets in log aggregators by applying a regex-based log masking filter:

import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class SecurityLogSanitizer {
private static final Pattern SENSITIVE_PATTERN = Pattern.compile(
"(Bearer\s+|bot|token=|xoxb-|xoxp-)[A-Za-z0-9_\-\.~/]+",
Pattern.CASE_INSENSITIVE
);

```
    public static String sanitize(String rawLog) {
        if (rawLog == null) return null;
        Matcher matcher = SENSITIVE_PATTERN.matcher(rawLog);
```

```
        return matcher.replaceAll("$1[REDACTED_SECRET]");
    }
```

}

1. Webhook HMAC Signature Verification
   Always verify incoming webhook payloads (interactive buttons, events) to prevent forgery and replay attacks.

import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;

public class HmacVerifier {

```
    // Verify Slack Requests (X-Slack-Signature)
    public static boolean verifySlackSignature(String signingSecret, String requestBody,
        // Prevent replay attacks (older than 5 minutes)
        if (Math.abs(System.currentTimeMillis() / 1000 - Long.parseLong(timestamp)) > 300
            return false;
        }
        String baseString = "v0:" + timestamp + ":" + requestBody;
        Mac mac = Mac.getInstance("HmacSHA256");
        mac.init(new SecretKeySpec(signingSecret.getBytes(StandardCharsets.UTF_8), "HmacS
        byte[] hash = mac.doFinal(baseString.getBytes(StandardCharsets.UTF_8));

        StringBuilder hexString = new StringBuilder("v0=");
        for (byte b : hash) {
            hexString.append(String.format("%02x", b));
        }
        return MessageDigest.isEqual(hexString.toString().getBytes(), signature.getBytes(
    }
```

}

1. Discord

Required Permissions Matrix
To perform E2E CRUD, DMs, User Invites, Threads, and Media actions, grant these OAuth2 Bot permissions:

Manage Channels -> MANAGE_CHANNELS (0x10)
Direct Messages -> Auto-enabled for shared servers
Channel Messages -> SEND_MESSAGES (0x800)
Threads -> CREATE_PUBLIC_THREADS (0x800000000)
User Invites -> CREATE_INSTANT_INVITE (0x1)
File Uploads -> ATTACH_FILES (0x8000)
Update/Delete -> MANAGE_MESSAGES (0x2000)

Java E2E Integration Test

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class DiscordE2ETest {
private static final String BOT_TOKEN = System.getenv("DISCORD_BOT_TOKEN");
private static final String GUILD_ID = System.getenv("DISCORD_GUILD_ID");
private static final String TARGET_USER_ID = System.getenv("DISCORD_TARGET_USER_ID");
private static final HttpClient client = HttpClient.newHttpClient();

```
public static void main(String[] args) throws Exception {
    // 1. CREATE CHANNEL
    String channelId = createChannel("e2e-discord-room");

    // 2. CREATE INVITE CODE
    String inviteCode = createChannelInvite(channelId);

    // 3. SEND DIRECT MESSAGE (DM)
    String dmChannelId = createDMChannel(TARGET_USER_ID);
    String dmMessageId = sendMessage(dmChannelId, "Hello via Direct Message!");

    // 4. SEND CHANNEL MESSAGE
    String msgId = sendMessage(channelId, "Hello Channel!");

    // 5. THREAD REPLY
    String threadId = createThreadAndReply(channelId, msgId, "Thread Reply Text");

    // 6. UPDATE & FILE UPLOAD
    updateMessage(channelId, msgId, "Updated Message Text");
    uploadFile(channelId, "test.txt", "File payload content");

    // 7. CLEANUP
    deleteMessage(channelId, msgId);
    deleteChannel(channelId);
```

```
}

private static String createChannel(String name) throws Exception {
    String json = String.format("{\"name\": \"%s\", \"type\": 0}", name);
    HttpRequest req = HttpRequest.newBuilder()
            .uri(URI.create("https://discord.com/api/v10/guilds/" + GUILD_ID + "/chan
            .header("Authorization", "Bot " + BOT_TOKEN)
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(json)).build();
    return extractValue(client.send(req, HttpResponse.BodyHandlers.ofString()).body()
}

private static String createChannelInvite(String channelId) throws Exception {
    String json = "{\"max_age\": 86400, \"max_uses\": 1}";
    HttpRequest req = HttpRequest.newBuilder()
            .uri(URI.create("https://discord.com/api/v10/channels/" + channelId + "/i
            .header("Authorization", "Bot " + BOT_TOKEN)
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(json)).build();
    return extractValue(client.send(req, HttpResponse.BodyHandlers.ofString()).body()
}

private static String createDMChannel(String userId) throws Exception {
    String json = String.format("{\"recipient_id\": \"%s\"}", userId);
    HttpRequest req = HttpRequest.newBuilder()
            .uri(URI.create("https://discord.com/api/v10/users/@me/channels"))
            .header("Authorization", "Bot " + BOT_TOKEN)
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(json)).build();
    return extractValue(client.send(req, HttpResponse.BodyHandlers.ofString()).body()
}

private static String sendMessage(String channelId, String text) throws Exception {
    String json = String.format("{\"content\": \"%s\"}", text);
    HttpRequest req = HttpRequest.newBuilder()
            .uri(URI.create("https://discord.com/api/v10/channels/" + channelId + "/m
            .header("Authorization", "Bot " + BOT_TOKEN)
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(json)).build();
    return extractValue(client.send(req, HttpResponse.BodyHandlers.ofString()).body()
}

private static String createThreadAndReply(String channelId, String msgId, String tex
    String json = "{\"name\": \"E2E-Thread\", \"auto_archive_duration\": 60}";
    HttpRequest req = HttpRequest.newBuilder()
            .uri(URI.create("https://discord.com/api/v10/channels/" + channelId + "/m
            .header("Authorization", "Bot " + BOT_TOKEN)
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(json)).build();
```

```
        String threadId = extractValue(client.send(req, HttpResponse.BodyHandlers.ofStrin
        sendMessage(threadId, text);
        return threadId;
    }

    private static void updateMessage(String channelId, String msgId, String newText) thr
        String json = String.format("{\"content\": \"%s\"}", newText);
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("https://discord.com/api/v10/channels/" + channelId + "/m
                .header("Authorization", "Bot " + BOT_TOKEN)
                .header("Content-Type", "application/json")
                .method("PATCH", HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void uploadFile(String channelId, String filename, String content) thr
        String boundary = "----Discord" + System.currentTimeMillis();
        String body = "--" + boundary + "\r\n" +
                "Content-Disposition: form-data; name=\"files[0]\"; filename=\"" + filena
                "Content-Type: text/plain\r\n\r\n" + content + "\r\n--" + boundary + "--\
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("https://discord.com/api/v10/channels/" + channelId + "/m
                .header("Authorization", "Bot " + BOT_TOKEN)
                .header("Content-Type", "multipart/form-data; boundary=" + boundary)
                .POST(HttpRequest.BodyPublishers.ofString(body)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteMessage(String channelId, String msgId) throws Exception {
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("https://discord.com/api/v10/channels/" + channelId + "/m
                .header("Authorization", "Bot " + BOT_TOKEN).DELETE().build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteChannel(String channelId) throws Exception {
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("https://discord.com/api/v10/channels/" + channelId))
                .header("Authorization", "Bot " + BOT_TOKEN).DELETE().build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static String extractValue(String json, String key) {
        String p = "\"" + key + "\":\"";
        int s = json.indexOf(p);
        if (s == -1) {
            p = "\"" + key + "\":";
            s = json.indexOf(p);
            if (s == -1) return "";
```

```
            s += p.length();
            int e = json.indexOf(",", s);
            if (e == -1) e = json.indexOf("}", s);
            return json.substring(s, e).trim();
        }
        s += p.length();
        int e = json.indexOf("\"", s);
        return json.substring(s, e);
    }
```

}

1. Telegram

Required Permissions Matrix

Topics/Channels -> createForumTopic, closeForumTopic (Requires Supergroup with Forum Topics enabled)
Direct Messaging -> sendMessage (Target user must initiate /start with bot first)
Group Messaging -> sendMessage (Bot must be added to Group)
File Uploads -> sendDocument, sendPhoto (Multipart form transmission)
Updates/Deletions -> editMessageText, deleteMessage (Bot can only edit its own messages)

Java E2E Integration Test

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class TelegramE2ETest {
private static final String BOT_TOKEN = System.getenv("TELEGRAM_BOT_TOKEN");
private static final String SUPERGROUP_ID = System.getenv("TELEGRAM_SUPERGROUP_ID");
private static final String DIRECT_USER_CHAT_ID =
System.getenv("TELEGRAM_USER_CHAT_ID");
private static final HttpClient client = HttpClient.newHttpClient();

```
    public static void main(String[] args) throws Exception {
        // 1. CREATE TOPIC
        String topicId = createTopic(SUPERGROUP_ID, "E2E Test Topic");

        // 2. SEND DIRECT MESSAGE TO USER
        String dmMsgId = sendMessage(DIRECT_USER_CHAT_ID, null, "Hello Direct Message Use
```

```
        // 3. SEND GROUP MESSAGE IN TOPIC
        String groupMsgId = sendMessage(SUPERGROUP_ID, topicId, "Hello Group Topic!");

        // 4. EDIT MESSAGE & UPLOAD DOCUMENT
        editMessage(SUPERGROUP_ID, groupMsgId, "Updated Group Topic Message");
        sendDocument(SUPERGROUP_ID, topicId, "report.pdf", "Sample PDF Data");

        // 5. CLEANUP
        deleteMessage(SUPERGROUP_ID, groupMsgId);
        closeTopic(SUPERGROUP_ID, topicId);
    }

    private static String createTopic(String chatId, String name) throws Exception {
        String url = "https://api.telegram.org/bot" + BOT_TOKEN + "/createForumTopic";
        String json = String.format("{\"chat_id\": \"%s\", \"name\": \"%s\"}", chatId, na
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body(),
    }

    private static String sendMessage(String chatId, String threadId, String text) throws
        String url = "https://api.telegram.org/bot" + BOT_TOKEN + "/sendMessage";
        String json = threadId == null ?
                String.format("{\"chat_id\": \"%s\", \"text\": \"%s\"}", chatId, text) :
                String.format("{\"chat_id\": \"%s\", \"message_thread_id\": %s, \"text\":
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body(),
    }

    private static void editMessage(String chatId, String msgId, String newText) throws E
        String url = "https://api.telegram.org/bot" + BOT_TOKEN + "/editMessageText";
        String json = String.format("{\"chat_id\": \"%s\", \"message_id\": %s, \"text\":
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void sendDocument(String chatId, String threadId, String filename, Str
        String url = "https://api.telegram.org/bot" + BOT_TOKEN + "/sendDocument";
        String boundary = "----TG" + System.currentTimeMillis();
        String body = "--" + boundary + "\r\nContent-Disposition: form-data; name=\"chat_
        if (threadId != null) {
            body += "--" + boundary + "\r\nContent-Disposition: form-data; name=\"message
        }
```

```
        body += "--" + boundary + "\r\nContent-Disposition: form-data; name=\"document\";

        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Content-Type", "multipart/form-data; boundary=" + boundary)
                .POST(HttpRequest.BodyPublishers.ofString(body)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteMessage(String chatId, String msgId) throws Exception {
        String url = "https://api.telegram.org/bot" + BOT_TOKEN + "/deleteMessage";
        String json = String.format("{\"chat_id\": \"%s\", \"message_id\": %s}", chatId,
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void closeTopic(String chatId, String threadId) throws Exception {
        String url = "https://api.telegram.org/bot" + BOT_TOKEN + "/closeForumTopic";
        String json = String.format("{\"chat_id\": \"%s\", \"message_thread_id\": %s}", c
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static String extractVal(String json, String key) {
        String p = "\"" + key + "\":";
        int s = json.indexOf(p);
        if (s == -1) return "";
        s += p.length();
        int e = json.indexOf(",", s);
        if (e == -1) e = json.indexOf("}", s);
        return json.substring(s, e).trim();
    }
```

}

1. Slack

Required Permissions Matrix (Bot Token Scopes)

Create Channels -> channels:manage (Create public channels)
Add Users to Channel -> channels:manage, groups:write (Invite workspace users to channel)
Direct Messages (1-on-1) -> im:write, mpim:write (Open 1-on-1 direct message conversation)
Send Messages -> chat:write (Post messages to channels and DMs)
Thread Replies -> chat:write (Post thread responses using thread_ts)

Update/Delete -> chat:write (Modify or remove posted messages)
Upload Files -> files:write (Upload media files to Slack conversations)

Java E2E Integration Test

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class SlackE2ETest {
private static final String BOT_TOKEN = System.getenv("SLACK_BOT_TOKEN");
private static final String TARGET_USER_ID = System.getenv("SLACK_TARGET_USER_ID");
private static final HttpClient client = HttpClient.newHttpClient();

```
public static void main(String[] args) throws Exception {
    // 1. CREATE PUBLIC CHANNEL
    String channelId = createChannel("e2e-slack-room");

    // 2. INVITE USER TO CHANNEL
    inviteUserToChannel(channelId, TARGET_USER_ID);

    // 3. OPEN DM CONVERSATION
    String dmChannelId = openDirectMessageChannel(TARGET_USER_ID);
    String dmTs = sendMessage(dmChannelId, "Hello via Slack DM!", null);

    // 4. SEND CHANNEL MESSAGE & THREAD REPLY
    String msgTs = sendMessage(channelId, "Hello Channel!", null);
    String replyTs = sendMessage(channelId, "Thread Reply Text", msgTs);

    // 5. UPDATE & FILE UPLOAD
    updateMessage(channelId, msgTs, "Updated Message Text");
    uploadFile(channelId, "log.txt", "Sample Log File Content");

    // 6. CLEANUP
    deleteMessage(channelId, msgTs);
    archiveChannel(channelId);
}

private static String createChannel(String name) throws Exception {
    String json = String.format("{\"name\": \"%s\"}", name);
    HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://slack.com/api/
            .header("Authorization", "Bearer " + BOT_TOKEN)
            .header("Content-Type", "application/json; charset=utf-8")
            .POST(HttpRequest.BodyPublishers.ofString(json)).build();
    return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body(),
```

```
}

private static void inviteUserToChannel(String channelId, String userId) throws Excep
    String json = String.format("{\"channel\": \"%s\", \"users\": \"%s\"}", channelId
    HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://slack.com/api/
            .header("Authorization", "Bearer " + BOT_TOKEN)
            .header("Content-Type", "application/json; charset=utf-8")
            .POST(HttpRequest.BodyPublishers.ofString(json)).build();
    client.send(req, HttpResponse.BodyHandlers.ofString());
}

private static String openDirectMessageChannel(String userId) throws Exception {
    String json = String.format("{\"users\": \"%s\"}", userId);
    HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://slack.com/api/
            .header("Authorization", "Bearer " + BOT_TOKEN)
            .header("Content-Type", "application/json; charset=utf-8")
            .POST(HttpRequest.BodyPublishers.ofString(json)).build();
    return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body(),
}

private static String sendMessage(String channelId, String text, String threadTs) thr
    String json = threadTs == null ?
            String.format("{\"channel\": \"%s\", \"text\": \"%s\"}", channelId, text)
            String.format("{\"channel\": \"%s\", \"text\": \"%s\", \"thread_ts\": \"%
    HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://slack.com/api/
            .header("Authorization", "Bearer " + BOT_TOKEN)
            .header("Content-Type", "application/json; charset=utf-8")
            .POST(HttpRequest.BodyPublishers.ofString(json)).build();
    return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body(),
}

private static void updateMessage(String channelId, String ts, String newText) throws
    String json = String.format("{\"channel\": \"%s\", \"ts\": \"%s\", \"text\": \"%s
    HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://slack.com/api/
            .header("Authorization", "Bearer " + BOT_TOKEN)
            .header("Content-Type", "application/json; charset=utf-8")
            .POST(HttpRequest.BodyPublishers.ofString(json)).build();
    client.send(req, HttpResponse.BodyHandlers.ofString());
}

private static void uploadFile(String channelId, String filename, String content) thr
    String boundary = "----Slack" + System.currentTimeMillis();
    String body = "--" + boundary + "\r\nContent-Disposition: form-data; name=\"chann
            "--" + boundary + "\r\nContent-Disposition: form-data; name=\"content\"\r
            "--" + boundary + "\r\nContent-Disposition: form-data; name=\"filename\"\
    HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://slack.com/api/
            .header("Authorization", "Bearer " + BOT_TOKEN)
            .header("Content-Type", "multipart/form-data; boundary=" + boundary)
            .POST(HttpRequest.BodyPublishers.ofString(body)).build();
```

```
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteMessage(String channelId, String ts) throws Exception {
        String json = String.format("{\"channel\": \"%s\", \"ts\": \"%s\"}", channelId, t
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://slack.com/api/
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "application/json; charset=utf-8")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void archiveChannel(String channelId) throws Exception {
        String json = String.format("{\"channel\": \"%s\"}", channelId);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://slack.com/api/
                .header("Authorization", "Bearer " + BOT_TOKEN)
                .header("Content-Type", "application/json; charset=utf-8")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static String extractVal(String json, String key) {
        String p = "\"" + key + "\":\"";
        int s = json.indexOf(p);
        if (s == -1) return "";
        s += p.length();
        int e = json.indexOf("\"", s);
        return json.substring(s, e);
    }
```

}

1. Google Chat

Required OAuth2 Scopes Matrix

Manage Spaces -> https://www.googleapis.com/auth/chat.spaces (Create/Delete Google Chat Spaces)
Direct Messages -> https://www.googleapis.com/auth/chat.memberships (Setup 1-on-1 space with a user)
Send/Update Messages -> https://www.googleapis.com/auth/chat.messages (Full message lifecycle & threading)

Java E2E Integration Test

import java.net.URI;
import java.net.http.HttpClient;

```
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class GoogleChatE2ETest {
private static final String ACCESS_TOKEN =
System.getenv("GOOGLE_CHAT_OAUTH_TOKEN");
private static final String TARGET_USER_EMAIL =
System.getenv("GOOGLE_TARGET_USER_EMAIL");
private static final HttpClient client = HttpClient.newHttpClient();
```

```
public static void main(String[] args) throws Exception {
    // 1. CREATE SPACE
    String spaceName = createSpace("E2E Test Space");

    // 2. ADD USER MEMBER TO SPACE
    addMemberToSpace(spaceName, TARGET_USER_EMAIL);

    // 3. SEND MESSAGE & THREAD REPLY
    String msgName = sendMessage(spaceName, "Hello Google Chat!", null);
    String threadReplyName = sendMessage(spaceName, "Thread reply content", msgName);

    // 4. UPDATE MESSAGE
    updateMessage(msgName, "Updated Message Text");

    // 5. CLEANUP
    deleteMessage(msgName);
    deleteSpace(spaceName);
}

private static String createSpace(String displayName) throws Exception {
    String json = String.format("{\"spaceType\": \"SPACE\", \"displayName\": \"%s\"}"
    HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://chat.googleapi
            .header("Authorization", "Bearer " + ACCESS_TOKEN)
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(json)).build();
    return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body(),
}

private static void addMemberToSpace(String spaceName, String userEmail) throws Excep
    String json = String.format("{\"member\": {\"name\": \"users/%s\", \"type\": \"HU
    HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://chat.googleapi
            .header("Authorization", "Bearer " + ACCESS_TOKEN)
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(json)).build();
    client.send(req, HttpResponse.BodyHandlers.ofString());
}
```

```
  private static String sendMessage(String spaceName, String text, String threadParentN
      String url = "https://chat.googleapis.com/v1/" + spaceName + "/messages";
      String json;
      if (threadParentName != null) {
          json = String.format("{\"text\": \"%s\", \"thread\": {\"name\": \"%s\"}}", te
          url += "?messageReplyOption=REPLY_MESSAGE_FALLBACK_TO_NEW_THREAD";
      } else {
          json = String.format("{\"text\": \"%s\"}", text);
      }
      HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
              .header("Authorization", "Bearer " + ACCESS_TOKEN)
              .header("Content-Type", "application/json")
              .POST(HttpRequest.BodyPublishers.ofString(json)).build();
      return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body(),
  }

  private static void updateMessage(String msgName, String newText) throws Exception {
      String json = String.format("{\"text\": \"%s\"}", newText);
      HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://chat.googleapi
              .header("Authorization", "Bearer " + ACCESS_TOKEN)
              .header("Content-Type", "application/json")
              .method("PATCH", HttpRequest.BodyPublishers.ofString(json)).build();
      client.send(req, HttpResponse.BodyHandlers.ofString());
  }

  private static void deleteMessage(String msgName) throws Exception {
      HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://chat.googleapi
              .header("Authorization", "Bearer " + ACCESS_TOKEN).DELETE().build();
      client.send(req, HttpResponse.BodyHandlers.ofString());
  }

  private static void deleteSpace(String spaceName) throws Exception {
      HttpRequest req = HttpRequest.newBuilder().uri(URI.create("https://chat.googleapi
              .header("Authorization", "Bearer " + ACCESS_TOKEN).DELETE().build();
      client.send(req, HttpResponse.BodyHandlers.ofString());
  }

  private static String extractVal(String json, String key) {
      String p = "\"" + key + "\": \"";
      int s = json.indexOf(p);
      if (s == -1) return "";
      s += p.length();
      int e = json.indexOf("\"", s);
      return json.substring(s, e);
  }
```

}

1. Microsoft Teams (Microsoft Graph API)

Detailed Permission Matrix (Delegated vs. Application)

For backend daemons acting without an active user session, Application Permissions with Admin Consent are mandatory.

Channels -> Create Channel -> Channel.Create, Group.ReadWrite.All (Standard Application Scope)
User Management -> Add/Remove Members -> TeamMember.ReadWrite.All (Manages roster in Team/Channel)
Direct Messages -> Create 1-on-1 or Group Chat -> Chat.Create, User.Read.All (Protected API - Requires Microsoft Form)
Direct Messages -> Send DM -> ChatMessage.Send (Protected API)
Channels -> Send Channel Message -> ChannelMessage.Send (Protected API)
Files -> Upload Attachments -> Files.ReadWrite.All, Sites.ReadWrite.All (Accesses SharePoint channel drive)

Setup Instructions

Simple Setup (Client Secret)

1. Go to Azure Portal -> Entra ID -> App registrations -> New registration.
2. Under Certificates & secrets, create a New client secret.
3. Under API permissions, grant required Graph permissions and click Grant admin consent for [Tenant].

Hardened Setup (Certificate & RSA Private Key)

1. Generate the RSA Private Key and Certificate via OpenSSL (as detailed in Section 1).
2. Go to your App Registration in Azure Portal -> Certificates & secrets -> Certificates -> Upload public_cert.pem.
3. In Java, generate the client assertion JWT using your local private_key.pem and request tokens via:
   POST https://login.microsoftonline.com/{tenant_id}/oauth2/v2.0/token with client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer.

Java E2E Integration Test

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class MSTeamsE2ETest {
private static final String ACCESS_TOKEN = System.getenv("MS_GRAPH_ACCESS_TOKEN");
```

```
private static final String TEAM_ID = System.getenv("MS_TEAMS_TEAM_ID");
private static final String TARGET_USER_ID =
System.getenv("MS_TEAMS_TARGET_USER_ID");
private static final HttpClient client = HttpClient.newHttpClient();
```

```
public static void main(String[] args) throws Exception {
    // 1. CREATE CHANNEL
    String channelId = createChannel(TEAM_ID, "E2E Test Channel");

    // 2. ADD USER TO CHANNEL
    addMemberToChannel(TEAM_ID, channelId, TARGET_USER_ID);

    // 3. CREATE DIRECT MESSAGE CHAT (1-on-1)
    String chatId = createDirectChat(TARGET_USER_ID);
    String dmMsgId = sendChatMessage(chatId, "Hello via Teams Direct Message!");

    // 4. SEND CHANNEL MESSAGE & THREAD REPLY
    String msgId = sendChannelMessage(TEAM_ID, channelId, "Hello Teams Channel!");
    String replyId = sendThreadReply(TEAM_ID, channelId, msgId, "Thread Reply Text");

    // 5. UPDATE MESSAGE
    updateChannelMessage(TEAM_ID, channelId, msgId, "Updated Teams Channel Message");

    // 6. CLEANUP
    deleteChannelMessage(TEAM_ID, channelId, msgId);
    deleteChannel(TEAM_ID, channelId);
}

private static String createChannel(String teamId, String name) throws Exception {
    String url = "https://graph.microsoft.com/v1.0/teams/" + teamId + "/channels";
    String json = String.format("{\"displayName\": \"%s\", \"membershipType\": \"stan
    HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
            .header("Authorization", "Bearer " + ACCESS_TOKEN)
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(json)).build();
    return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body(),
}

private static void addMemberToChannel(String teamId, String channelId, String userId
    String url = "https://graph.microsoft.com/v1.0/teams/" + teamId + "/channels/" +
    String json = String.format("{\n" +
        "    \"@odata.type\": \"#microsoft.graph.aadUserConversationMember\",\n" +
        "    \"roles\": [],\n" +
        "    \"user@odata.bind\": \"https://graph.microsoft.com/v1.0/users('%s')\"\n"
        "}", userId);
    HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
```

```
            .header("Authorization", "Bearer " + ACCESS_TOKEN)
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(json)).build();
    client.send(req, HttpResponse.BodyHandlers.ofString());
}

private static String createDirectChat(String userId) throws Exception {
    String url = "https://graph.microsoft.com/v1.0/chats";
    String json = String.format("{\n" +
        "    \"chatType\": \"oneOnOne\",\n" +
        "    \"members\": [\n" +
        "        {\n" +
        "            \"@odata.type\": \"#microsoft.graph.aadUserConversationMember\",
        "            \"roles\": [\"owner\"],\n" +
        "            \"user@odata.bind\": \"https://graph.microsoft.com/v1.0/users('%
        "        }\n" +
        "    ]\n" +
        "}", userId);
    HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
            .header("Authorization", "Bearer " + ACCESS_TOKEN)
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(json)).build();
    return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body(),
}

private static String sendChatMessage(String chatId, String text) throws Exception {
    String url = "https://graph.microsoft.com/v1.0/chats/" + chatId + "/messages";
    String json = String.format("{\"body\": {\"content\": \"%s\"}}", text);
    HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
            .header("Authorization", "Bearer " + ACCESS_TOKEN)
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(json)).build();
    return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body(),
}

private static String sendChannelMessage(String teamId, String channelId, String text
    String url = "https://graph.microsoft.com/v1.0/teams/" + teamId + "/channels/" +
    String json = String.format("{\"body\": {\"content\": \"%s\"}}", text);
    HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
            .header("Authorization", "Bearer " + ACCESS_TOKEN)
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(json)).build();
    return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body(),
}

private static String sendThreadReply(String teamId, String channelId, String msgId,
    String url = "https://graph.microsoft.com/v1.0/teams/" + teamId + "/channels/" +
    String json = String.format("{\"body\": {\"content\": \"%s\"}}", text);
    HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
```

```
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json)).build();
        return extractVal(client.send(req, HttpResponse.BodyHandlers.ofString()).body(),
    }

    private static void updateChannelMessage(String teamId, String channelId, String msgI
        String url = "https://graph.microsoft.com/v1.0/teams/" + teamId + "/channels/" +
        String json = String.format("{\"body\": {\"content\": \"%s\"}}", newText);
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .header("Content-Type", "application/json")
                .method("PATCH", HttpRequest.BodyPublishers.ofString(json)).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteChannelMessage(String teamId, String channelId, String msgI
        String url = "https://graph.microsoft.com/v1.0/teams/" + teamId + "/channels/" +
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN)
                .POST(HttpRequest.BodyPublishers.noBody()).build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static void deleteChannel(String teamId, String channelId) throws Exception {
        String url = "https://graph.microsoft.com/v1.0/teams/" + teamId + "/channels/" +
        HttpRequest req = HttpRequest.newBuilder().uri(URI.create(url))
                .header("Authorization", "Bearer " + ACCESS_TOKEN).DELETE().build();
        client.send(req, HttpResponse.BodyHandlers.ofString());
    }

    private static String extractVal(String json, String key) {
        String p = "\"" + key + "\":\"";
        int s = json.indexOf(p);
        if (s == -1) return "";
        s += p.length();
        int e = json.indexOf("\"", s);
        return json.substring(s, e);
    }

}
```